

第二十届全国大学生智能汽车竞赛

室外 5G 远程驾驶无人车赛项目

技术报告

学 校: 山东大学

队伍名称: 西格玛队

参赛队员: 杨瑞 孟时剑 郭宣辰 欧阳凌峰

带队老师: 房明

2025 年 12 月 2 日

关于技术报告和研究论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛关于保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和赞助公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：郭庭辰 欧阳凌峰 孟时剑 杨瑞

带队教师签名：房明

日期：12月1日

目录

1	方案概述	1
1.1	自主导航	1
1.1.1	技术实现路线	1
1.1.2	创新点	1
1.1.3	测试效果	2
2	问题描述	2
2.1	自主导航	2
2.2	语音播报	2
3	技术方案	3
3.1	数据获取方式创新点	3
3.2	语音交互创新点	3
3.3	自主导航的实现方式及创新点	3
3.3.1	程序开发策略	3
3.3.2	任务拆分策略	4
3.3.3	蓝色挡板识别创新点	5
3.3.4	车道线检测及巡线创新点	5
3.3.5	弯道锥桶识别和避障创新点	8
3.3.6	人行横道的检测和停车创新点	9
3.3.7	转向标志识别创新点	10
3.3.8	锥桶引导创新点	12
3.3.9	识别停车位创新点	12
4	方案实现	14
4.1	数据采集修正	14
4.2	提升程序编译效率	14
4.3	转向动态优化调整	15
4.4	多线程设计时内存竞争读写问题	15
4.5	巡线方案迭代	15
4.5.1	图像处理优化	16
4.5.2	引入坐标变换	17
4.5.3	优化控制算法	17
4.6	开发 HSV 颜色阈值调试工具	18
5	测试分析	19
5.1	蓝色挡板识别测试	19
5.2	斑马线识别模块	20
5.3	转向标志识别测试	20
5.4	锥桶识别模块	21

5.5 停车标志测试	22
6 作品总结	23
附录 Appendix	24

1 方案概述

1.1 自主导航

1.1.1 技术实现路线

本方案通过先进的图像处理和算法，实现无人车的自主导航。具体步骤如下：

- **图像标定与矫正：**通过棋盘格标定法对上方摄像头（简称 cam2，下同）进行校准，提取角点并生成内外参数，便于后续图像处理，提高巡线结果的准确率。
- **自主导航算法：**结合多线程编程，分别处理巡线、障碍物识别和停车标志检测等任务，确保各功能模块适时运行，提高整体效率。
- **巡线处理与控制：**通过对 cam2 中的图像进行矫正、裁剪、灰度变换等图像处理方法，应用霍夫直线检测提取直线。经过过滤和分类，获取符合特定角度和长度要求的线段，并根据斜率的正负将其区分为左边线和右边线。通过几何变换和对小车运行时的特殊假设，可一定程度上还原出小车在跑道的真实位置关系，然后通过非线性变换和微分算法，将小车速度方向的调整与小车舵机角度联系起来，达到控制小车在跑道内稳定行驶的目的。
- **锥桶识别及避障：**通过颜色识别，找寻最小外接圆，同时增加宽高比的限制，并根据全局光照动态调整蓝色阈值增强鲁棒性，并且引入最小识别次数防止误识别到干扰因素，增强锥桶识别的稳定性。
- **斑马线识别及语音播报：**斑马线识别部分沿用识别白线逻辑，设计识别阈值。处理语音交互问题时，采用讯飞 SDK 软件开发包完成语音合成。
- **转向标志识别：**先提取蓝色轮廓，然后采用矩形坐标矫正，将转向标志进行二值化、膨胀处理，然后从中线分开比较左右两边像素值大小，据此判断转向。为提高识别准确率和鲁棒性，采集连续六张图像进行识别，结果一致即确认转向方向。
- **锥桶引导返回原赛道：**完成变道之后恢复正常巡线状态；加入两个触发条件，即黄色锥桶大小满足最小阈值和蓝色锥桶个数减少到最小数值，进入锥桶引导阶段，沿规定方向返回原赛道。
- **停车标志识别：**根据 AB 图像连通区域个数不同进行区分。

1.1.2 创新点

- **双摄像头协同工作：**通过合理分配下方摄像头（简称 cam0）和云台摄像头（cam2）的功能，优化了图像采集与处理的效率，确保无人车在复杂环境中稳定运行。
- **巡线及人行横道识别处理：**通过提前测试矫正参数，解决了 cam2 带来的图像扭曲问题；处理人行横道和识别白线使用同一图像处理方法，通过灰度和二值化提高识别精度，降低光照和场地变色对结果带来的干扰；同时加入矩形滤波算法，有效去除场地跑道线、起跑线及其他因素的干扰，能够实现高精度的人行横道识别。
- **锥桶识别处理：**只根据找寻最小外接圆来判断锥桶很不稳定，容易受到光线及场地因素干扰，于是增加宽高比的限制，并根据全局光照动态调整蓝色阈值增强鲁棒性，并且引入最小识别次数防止误识别到干扰因素，增强锥桶识别的稳定性。

- **非线性控制算法：**通过改进 PID 控制方法，实现对车辆精确的导航与转向控制，使其在变化的环境中保持稳定。
- **多线程模块化管理：**多线程的设计完成了任务分解，将复杂任务分成小部分，便于管理和调试；可以提高性能，同时处理多个任务；有良好可扩展性，容易增加线程来处理更多请求；实现了内存和其他资源共享，节省开销。
- **二维码识别处理：**在最后扫码时，为了提高扫码成功率，再打开摄像头之后，利用程序处理并提取二维码，大大提高扫码成功率。
- **kick_stop 减速模块：**在高速行驶转换到低速行驶时，速度存在不稳定性，无法准确转换到要求的速度值，相当于“松油门”，会导致后续任务无法完美完成。于是引入 kick_stop 模块，在转换时极短时间内进行“刹停”，然后达到目标低速值，提高速度转换的稳定性。

1.1.3 测试效果

车辆在比赛过程中成功避开障碍物，准确识别斑马线、转向和停车标志，变道之后沿锥桶返回原赛道，并在停车区域内顺利停靠。

2 问题描述

2.1 自主导航

在无人驾驶部分，本文将任务分解为四个主要部分：巡线、避障、图像识别和速度控制。其中，图像识别部分包含蓝色挡板、人行横道、转向和停车位标志识别。

巡线部分的核心问题是“通过识别跑道两侧的白线，控制小车在跑道内稳定地行驶”。为此本文在图像处理、坐标系变换、PID 控制多方面进行了尝试和协调。

避障部分的难点是“在较高速情况和不同光照下，稳定快速识别锥桶并“绕过”。为此本文采集了不同光照下大量数据集，结合实地测试，不断改进速度、转向时刻和识别标准。

在图像识别部分，蓝色挡板的核心问题是“准确区分蓝色挡板和蓝色地面”，为此本文采用“取反”思路，识别非蓝色区域，提高判断的准确性；人行横道的核心问题是“如何准确识别人行横道，不被操场额外杂线干扰”，为此本文优化了霍夫直线检测部分的过滤条件；转向识别的核心问题是“如何快速准确识别转向标志，减少受每次在斑马线前停车位置的影响”，为此本文提出“坐标分析”的方法，高效判断转向，同时优化在斑马线前的速度控制和刹车，基本稳定每次的停车位置；停车位标志的核心问题是“如何在很小开销下快速区分标志 A 和 B”，为此本文提出了“连通区域检测”的方法，根据连通块个数判断 AB。

速度控制部分的难点在于“如何合理分配各段速度，使得整体耗时少且效果稳定”。为此本组实地进行了长时间测试，通过设置不同函数，分别测试起步、刹车、各段的最优速度。

2.2 语音播报

在语音播报部分，本文使用科大讯飞 SDK 实现了语音合成功能。赛事组提供的资料已经能够实现语音的合成和播放。因此，需要解决的关键性问题是将现有的编译功能组合起来，并控制播报时间在 3s 以内。故本文在语音合成后，对语音进行了声音放大和加速，确保其可以在斑马线停车时顺利播报。

3 技术方案

3.1 数据获取方式创新点

为了确保相机标定的准确性，我们选择了 26 张不同角度和距离下拍摄的棋盘格图像，目的是让棋盘格在图像中占据较大区域，以便于准确检测角点。在图像处理过程中，我们首先将棋盘格作为标定对象，从捕捉到的图像中提取角点。具体步骤是，首先读取每张图像并转为灰度图像，然后使用 `findChessboardCorners` 函数检测角点位置，并通过 `cornerSubPix` 函数进行亚像素级精确化，以提高角点的检测精度。接下来，为每个检测到的角点生成对应的三维世界坐标，并将这些角点和它们的世界坐标存储在 `imagePoints` 和 `objectPoints` 中。之后，我们使用 `calibrateCamera` 函数计算相机的内外参数，包括相机矩阵和畸变系数，同时通过计算出的 RMS 误差来评估标定的精度。最后，我们将相机的内参和畸变系数保存到 XML 文件中，以便于后续使用。

3.2 语音交互创新点

智能车的麦克风位于车头的摄像头 I 中，因此录出的音频不仅小，并且本文发现对于每段音频的开始部分存在小段尖峰脉冲干扰，对语音转文字的准确度造成了极大的干扰，为此本文提出采用中值滤波算法对尖峰脉冲干扰过滤，同时对过滤后的语音信号进行放大，以此保证识别的清晰和准确。

3.3 自主导航的实现方式及创新点

3.3.1 程序开发策略

我们选择使用 C++ 进行主要程序的开发，并采用多线程设计和多文件编译的方案。这样做的好处有三：其一，C++ 提供了更高的程序执行效率，运行时间开销更低，相比于同样的 Python 实现，C++ 不会存在单个 CPU 资源的浪费，能够显著提高程序的性能，这对低延时的图像处理任务尤为关键；其二，在我们的项目中，两个摄像头（cam0 和 cam2）分别用于完成不同的任务：cam0 专注于复杂图像处理，而 cam2 负责巡线。在此基础上，通过引入多线程的设计，可使两个任务几乎互不干扰地并行运行，从而提高整个 CPU 的利用率。具体来说，巡线任务独自占用一个线程，而斑马线识别、避障等其他任务则另一线程中运行，同时父进程负责调度。这样可以避免串行任务导致的各任务处理的周期变长，降低控制延时，提高整个 CPU 的利用率；其三，多文件编译方便各个独立的功能管理，每个文件可以着重实现对应的功能，一方面可以减少调试某个模块时编译的时间，另一方面方便团队合作和代码项目管理，合并代码时更方便调理。

3.3.2 任务拆分策略

由于环境的复杂性以及算力限制，自主导航全过程各功能，如锥桶绕行、人行横道检测等，需要分开实现，不需要同时对所有目标进行检测，实现难度极大的同时，对硬件要求也非常高。因此，考虑整体流程，将任务进行拆解，设置不同标志代表对应模式，在各状态之间进行转换，从而完成各任务，如图1所示：

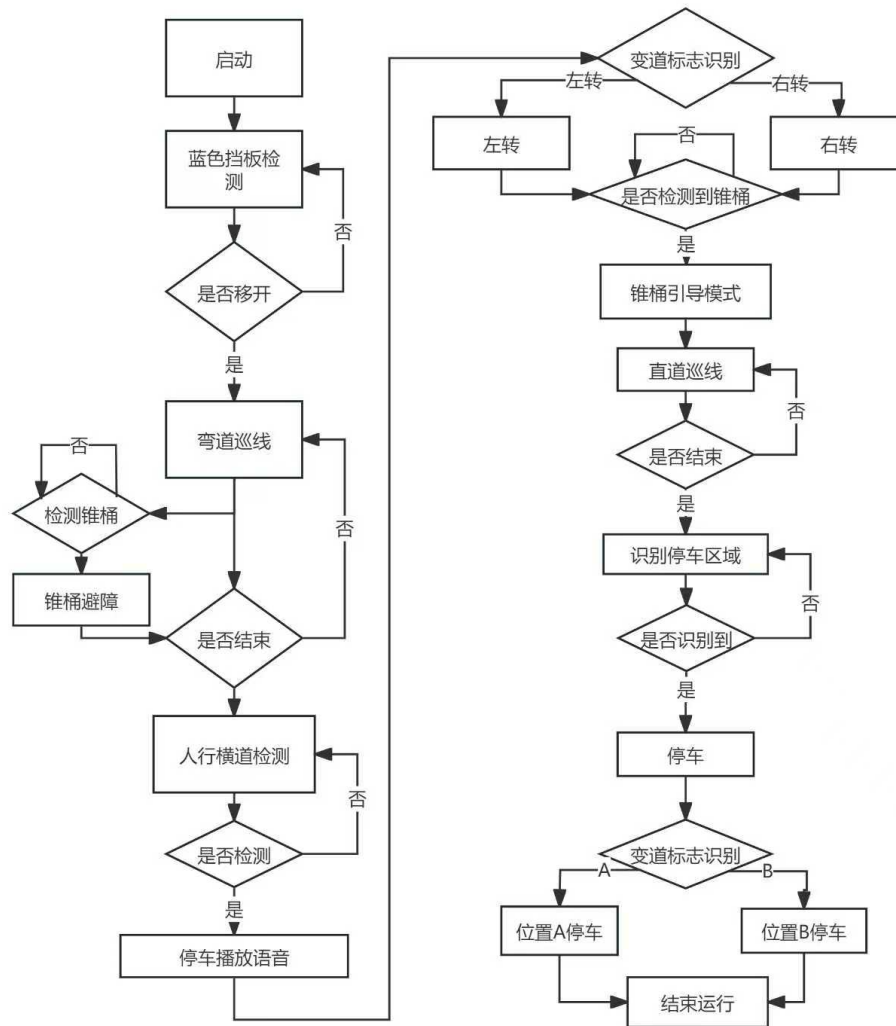


图 1: 全过程功能流程图

3.3.3 蓝色挡板识别创新点

在蓝色挡板的识别中，关键在于有效区分蓝色地面和蓝色挡板。为了解决这一问题，我们首先将输入图像从 BGR 色彩空间转换为 HSV 色彩空间，这样可以更精确地处理颜色信息。接着，我们使用阈值化操作，通过设定蓝色的上下限生成一个二值掩膜，从而提取出图像中所有蓝色区域。为了进一步提高提取结果的质量，我们应用形态学操作，如腐蚀和高斯模糊，这些处理能够减少噪声并平滑掩膜，确保蓝色区域的清晰度和连通性。在获取清晰的蓝色掩膜后，我们接下来计算掩膜中非蓝色像素的数量。通过使用 `countNonZero` 函数，我们可以快速得出蓝色像素的总数，然后用图像的总像素数减去这个值，从而获得非蓝色像素的数量。通过计算非蓝色像素在总像素中的比例，我们可以判断非蓝色区域的占比。如果这个比例超过预设的阈值，系统就会将该区域识别为蓝色挡板。这种方法通过专注于非蓝色区域的分析，能够有效区分蓝色地面与蓝色挡板，不仅提升了识别的准确性，还增强了系统在复杂环境中的鲁棒性。

3.3.4 车道线检测及巡线创新点

巡线的核心问题在于，通过有限的图像信息，控制小车向合适的方向前进。因此需要寻找一种稳定且高效的映射关系，通过给出的图像信息，得到小车车轮应该朝向的方向。而在这样的场景下，这样的映射非常容易构造。在此基础上引入 PID 控制的方案，比纯粹使用 PID 控制的方法更为稳定、高效。

图像二值化 考虑到对操场地面图像的识别核心在对不同深浅亮度蓝色的区分，选取分辨率更低 `cam2` 作为图像源，这样在保证整体识别效果稳定的前提下，可以减少图像读取和处理时造成的开销。先将拍摄到的画面进行如下处理，如图2所示：

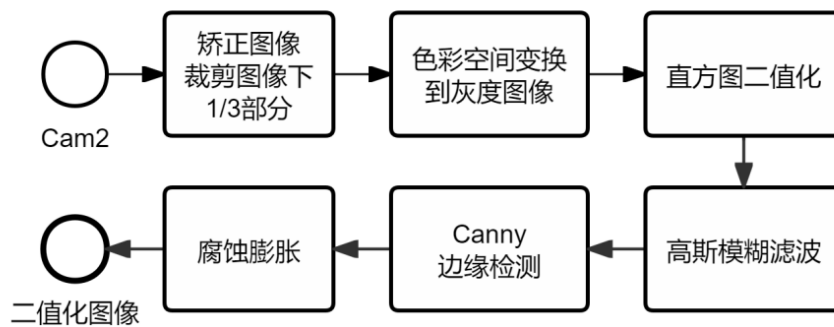


图 2: 巡线图片二值化流程

坐标变换 由于云台的旋转角度可以在两个轴向上进行调整，自动驾驶前可以自动校准云台到相对固定的位置，故我们可以假设 `cam2` 对准的方向以及相对于小车的相对位置保持不变。另外，通过对摄像头输出图像的矫正，可以认为地面平面空间可以由图像平面（屏幕平面空间）从摄像头原点作投影得到。在某一固定的坐标系下（实现时选取小车行进的方向为 y 轴，小车右侧方向为 x 轴），可以很容易得到以下几何关系，如图3所示：

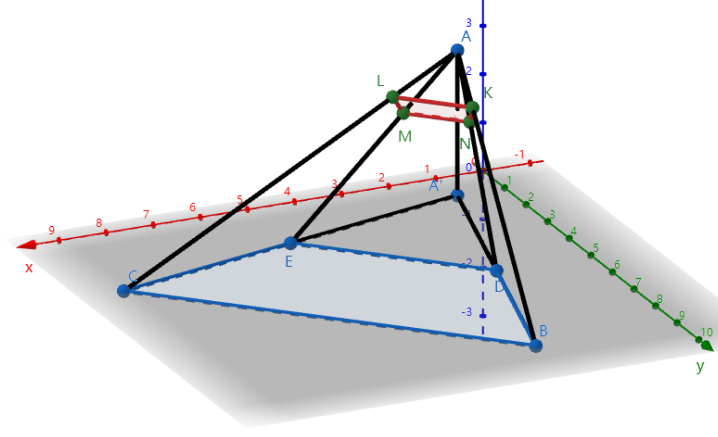


图 3: 坐标变换示意图

图3中, 从 A 点 (对应 cam2 的位置) 将矩形 KLMN (对应屏幕中线以下的下 $\frac{1}{4}$ 部分) 对地平面 xOy 投影, 得到梯形 BCED。通过比对 cam2 即时输出的图像, 我们对 B, C, D, E 四点在现实空间的位置进行了定位, 并且测量了相关数据:

$$h = |AA'|, d_n = \text{distance}(A, DE), d_f = \text{distance}(A, BC)$$

经过简单的几何推导, 得到了屏幕平面空间到地平面空间的坐标变换映射:

$$f(x, y) = (p(x), q(y))$$

$$p(x) = \sqrt{\frac{h^2 + q^2(y)}{d^2 + y^2}} \cdot (x - m), q(y) = h \cdot \tan(\alpha - \arctan(\frac{y}{d}))$$

在给定的 cam2 位置和 320x240 分辨率的图像中, 有以下常数关系:

$$m = 160, \alpha = \arccos(\frac{h}{d_f}), d = \frac{60}{\tan(\alpha - \arctan(\frac{h}{d_n}))}$$

为了减少每次图像处理坐标变换使用的时间, 我们提前将所有可能用到的坐标值保存到表中, 在程序开始时加载, 坐标变换时只需查表即可。实现时, 为了方便计算, 取 A' 为坐标原点。

将二值化之后的图像通过霍夫直线检测, 得到若干线段, 过滤掉其中较为水平和竖直的直线后, 求出各类平均直线的斜率和截距:

$$\vec{v} = \sum_{i=1}^n \vec{v}_i, m = \frac{\vec{v}_y}{\vec{v}_x}, b = \frac{\sum_{i=1}^n (y_i - \frac{\vec{v}_i y}{\vec{v}_i x})}{n}$$

其中 $\vec{v}_i$ 是霍夫直线检测结果中某一类线段两点之间向量, m, b 是直线 $y = mx + b$ 的参数, n 是求和线段个数。实际实现时, 发现有以下三个问题, 并提出了对应的解决方案:

1. 映射后的直线大部分情况下接近竖直, 此时 y 关于 x 方程的斜率会非常大, 此时可能会由于 double 类型精度不足导致计算误差。因此, 在映射后的直线采用 x 关于 y 的方程 $x = my + b$ 保存直线信息。

2. 由于小车的位置并不是时刻位于跑到中间，所以会出现只能捕捉到单侧白线的情况，此时需要通过跑道宽度（1.22m）推测另一条直线的位置。
3. 为了应对各种情况（图像采集不稳定的情况）并减少由于分支预测导致的时间开销，我们将处理不同情况的函数打包成函数数组，并通过霍夫直线检测的结果直接调用对应函数数组中的函数。

反馈控制 得到地平面 xOy 中两条白线的信息后，通过下图的方法得到小车速度的目标方向，如图4所示：

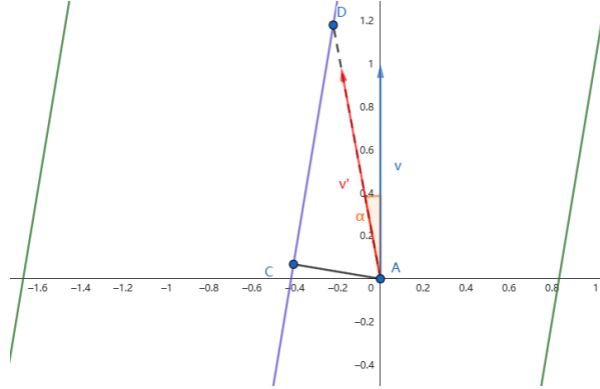


图 4: 目标方向获取示意图

根据上述建系方法， $\vec{v}$ 的方向即为小车当前的速度方向，取定一个 $|CD|$ 的值和目标阈值（用于控制小车位于跑道中横向位置，此处为 0.5），可以通过绿色直线（对应跑道白线）的位置推算出紫色直线（目标直线）的参数，进而计算得到 $|AC|$ 和 α 的大小，其中 α 即为舵机需要调整的角度。

$$\vec{v}' = \sqrt{\frac{b'^2}{1+m'^2} + |CD|^2} \cdot \vec{e} + (b', 0), \vec{e} = (m', 1)$$

$$b' = b_l + t \cdot (b_r - b_l), m' = m$$

其中 b_l, b_r, m 分别为左绿线截距、右绿线截距和绿线斜率（ $x = my + b$ ）；实际计算时 $\vec{AC}$ 发现的 y 坐标很小，所以近似为 0 处理简化计算。同时，实际实现中，我们发现合适的 $|CD|$ 与小车行驶速度有关，当电调的 PWM 值为 11500 时恰取 122.0。通过这样的设计，还可以通过修改目标阈值，控制小车在跑道的某一横向位置行驶。

巡线的整体解决方案如下（省略了异常处理）：

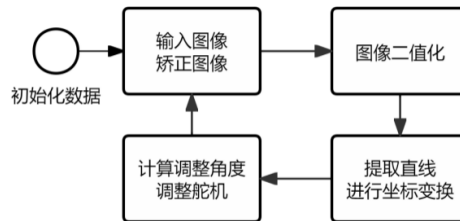


图 5: 巡线方案图

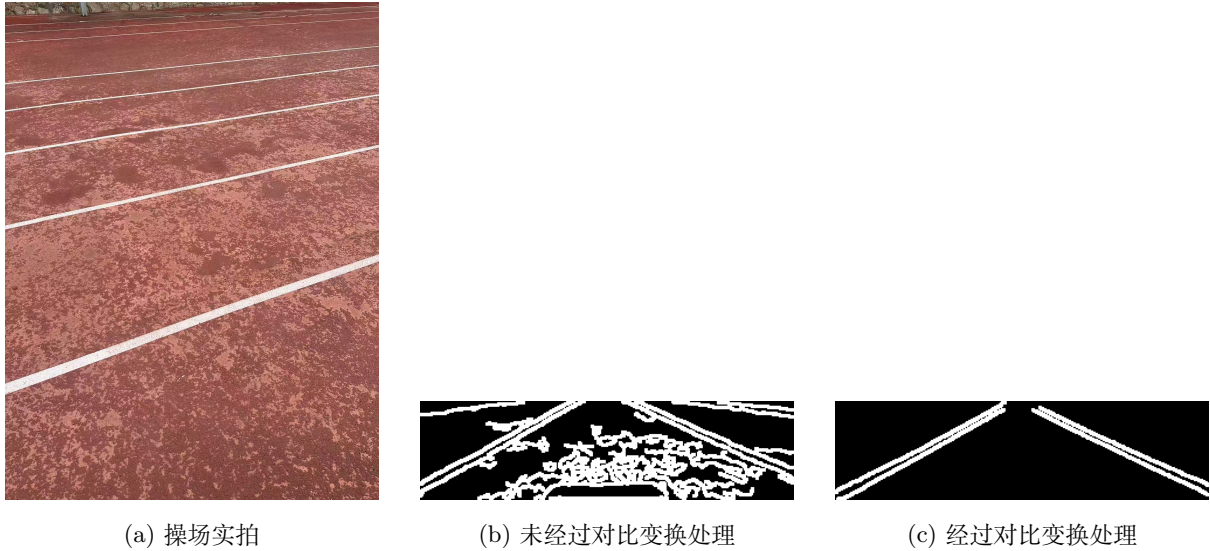


图 6: 对比变换

对比变换 如图6所示，由于操场年久失修，跑道中有许多噪点，加上光照、阴影等复杂现实情况干扰，包含大量无效信息，在处理时这些干扰信息会对车辆巡线稳定性产生巨大干扰，造成车辆巡线不稳，严重时甚至会导致车辆冲出赛道，导致比赛任务失败。

于是我们尝试在巡线代码中加入对比变换处理，即在遇到跑道颜色不均匀时，通过比对跑道颜色和边线阈值，更好的识别跑道边线，实现巡线的稳定性和鲁棒性，以适应不同的比赛环境。

通过测试，对比变换可以很好的处理跑道颜色不均匀的问题（如图6b、c 所示），在处理后二值化图像基本只保留跑道两侧边线，提高代码的适应性。

3.3.5 弯道锥桶识别和避障创新点

在识别锥桶部分，我们采取了简单高效的颜色识别。因为 HSV 范围对应红色和蓝色、白色等差别很大，所以可以通过简单测试加 HSV 空间，找到合适的蓝色阈值。根据亮暗两个阈值范围，先进行掩膜合并，然后进行腐蚀和高斯模糊，目的是去除噪声，改善边缘平滑度，以便于增强图像中锥桶的可识别性。起初，我们通过 `findcontours` 函数查找蓝色区域的轮廓，画最小外接圆并检查半径，根据半径阈值判断当前是否识别到了预期的一次锥桶。但是该方法很容易受到光照及场地其他干扰因素的影响，于是增加宽高比的限制，并根据全局光照动态调整蓝色阈值增强鲁棒性，并且引入最小识别次数防止误识别到干扰因素，增强锥桶识别的稳定性。

避障部分可以简单分为四个阶段：识别，减速，避障，恢复巡线。在识别到蓝色锥桶部分，执行 `kick_stop` 并同时完成避障操作，然后推出锥桶避障，恢复弯道巡线。

3.3.6 人行横道的检测和停车创新点

基本线程 在弯道巡线线程结束后，启动人行横道检测线程。在人行横道检测线程中，延续巡线的二值化处理，然后从二值图裁剪出中间区域作为 ROI，去掉左右和上下的干扰后进行轮廓分析，对每个轮廓计算面积与其外接矩形面积的比例，比例达到阈值则记作有效斑马线条块。累计有效条块数量，当达到最小条块数时返回检测成功。线程的工作流程如下，如图7所示：

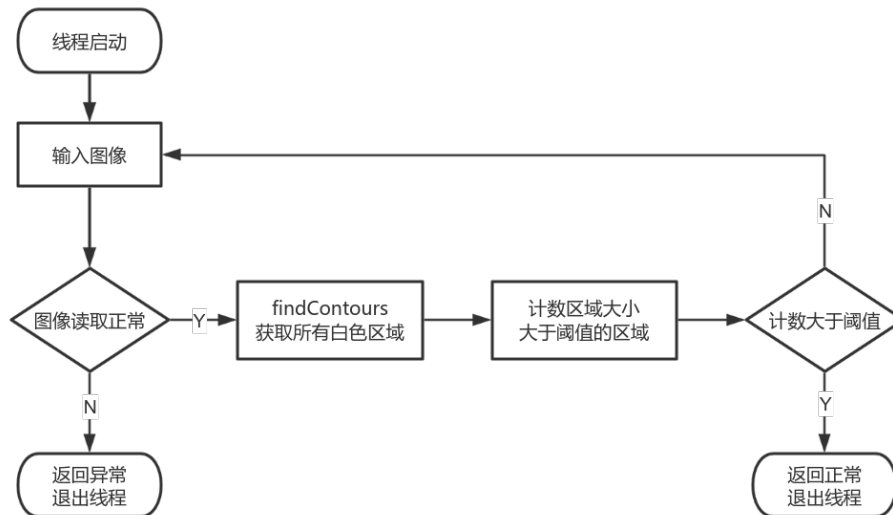


图 7: 斑马线检测流程图

矩形过滤 在实际运行过程中，二值化处理之后，图像中会有大量干扰色块被计算为斑马线，如跑道线，起跑线，地面数字标等等 (如图8 (a) 所示)，因为这些干扰因素无法避免且通过单纯限制色块大小无法有效解决，为了提高稳定性，我们想出矩形滤波算法来解决该问题。即填充比例 $fill_ratio$ 衡量轮廓在其外接矩形内的“实心程度”，目标是区分“斑马线条块”（接近实心的白条）与“细线/噪声/稀疏碎片”。对规则白条， $area$ 与 $rect_area$ 接近， $fill_ratio$ 高，容易通过；对细线或轮廓边界粗糙的噪声， $area$ 远小于 $rect_area$ ，比例低被过滤；对非常不规则或断裂的连通域，比例通常偏低，也会被过滤。如图8 (b) 所示



(a) 未加矩形滤波（跑道线等会被计算为有效面积）



(b) 加矩形滤波

图 8: 矩形滤波处理

线程切换 主线程调度子线程的方式则是如图9所示的方式：

实际实现中，我们取区域大小阈值为 50-60，计数器阈值为 6；同时，为了减少不必要的线程间通信，这里选择使用 `join (pthread_join)` 阻塞主线程来实现等待子线程退出的效果。

其他子任务（子线程）的调度方式类似，故不再赘述。

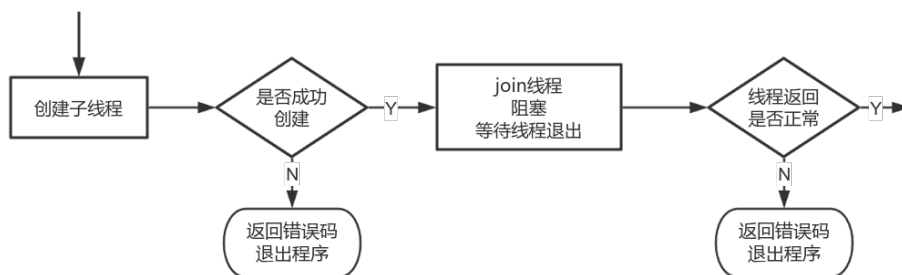


图 9: 斑马线检测流程图

3.3.7 转向标志识别创新点

变道标志识别 为了让小车快速找到箭头敏感区域，我们让识别范围局限在蓝色矩形里，根据蓝色矩形上半部分区域里的白色像素点左右空间占比来判定方向。流程如图10所示：

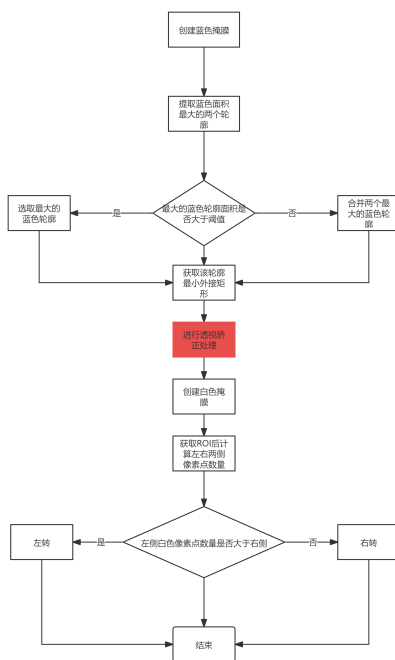
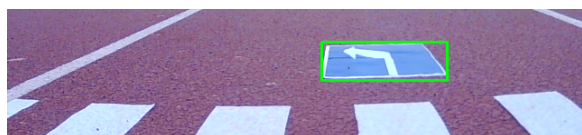


图 10: 箭头识别

敏感区域提取 识别时，为了让智能车首先找到变道标志区域，采用蓝色蒙版，找到提取出对应区域但是在多次测试发现检测到的蓝色最大轮廓不是一个完整的标志符，这是由于箭头的下半部分和蓝色标志符带有的一些白边在比较平的俯角下识别为一个整体，这样就会把一个完整的蓝色区域分割开来，经过多次测试发现识别完整的蓝色轮廓至少需要 1250（池化以后的红色像素点，当检测出的最大的蓝色轮廓不满足这个阈值时，将最大的两个蓝色轮廓合并时，就可以提取出我们需要的范围。

透视矫正 在人行横道前停车时，无法保证每次都停在跑道正中央，如果小车停车位置有所偏差，会导致提取蓝色区域产生畸变，造成识别有所误差，导致无法准确识别转向标志，于是我们加入透视校正算法，用透视变换把倾斜/透视下的箭头规范到正视矩形，便于后续左右半区的白像素计数。具体线程：蓝色区域分割 → 轮廓合并与凸包 → 四边形近似 → 点排序 → 构建透视矩阵 → 输出标准化图像。经过透视矫正（如图11所示）之后，箭头方向信息在标准化视图中更居中且姿态统一，左右半区像素计数更稳定，识别准确率更高。



(a) 蓝色区域分割



(b) 透视矫正

图 11: 透视矫正

3.3.8 锥桶引导创新点

本任务的难点是如何在巡线过程中准确识别出引导区域，按锥桶引导返回原赛道，于是我们加入两个触发条件，即黄色锥桶大小满足最小阈值和蓝色锥桶个数减少到最小数值，进入锥桶引导阶段，沿规定方向返回原赛道。具体线程如图12所示：

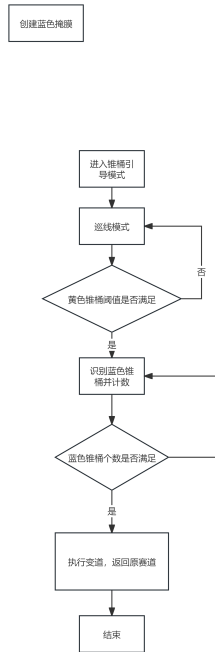


图 12: 锥桶变道线程

3.3.9 识别停车位创新点

本任务的难点是如何准确识别到字母标志牌。由于无人车在高速行驶时，摄像头采集到的画面画质较低，无法满足准确识别的要求。为此本文采用分段减速策略，无人车通过提取字母标志牌的颜色判断无人车与停车区域的距离。在识别到蓝色标志区域大小满足阈值与宽高比限制时（如图13所示），会减速停车，并开始字母标识牌的识别，该策略保证了字母标识牌的准确识别。



(a) 停车区域较远

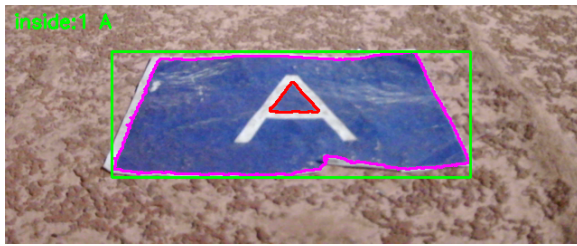


(b) 停车区域较近

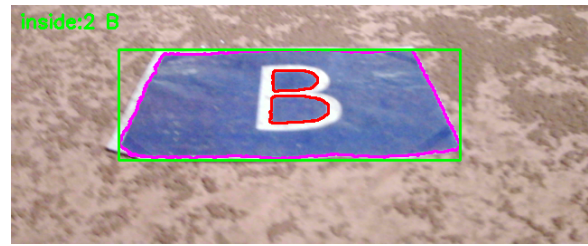
图 13: 停车区域检测

在识别方面，通过观察发现，字母“A”与字母“B”在提取符合阈值的掩膜后，最大的区别在于其中的连通区域，其中字母“A”拥有一个连通区域，而字母“B”拥有两个连通区域。为尽可能

保证连通区域的识别不被噪声干扰，我们对图像进行了裁剪，仅对图像中的中间部分进行识别。我们提取图像中符合 HSV 阈值的区域，通过最大连通区域中的连通区域的个数判断该字母标识牌的种类。如图14所示，红色为内部轮廓：



(a) 标识 A 轮廓层级图



(b) 标识 B 轮廓层级图

图 14: 停车标志识别

4 方案实现

4.1 数据采集修正

在实地测试时发现校园操场有坡度，而且 cam0 的相机朝向角度不稳定，故用 cam0 采集的数据做巡线处理很困难。考虑到图像大小直接与处理开销挂钩，选择上面摄像头 cam2 进行巡线是更明智的选择。在锥桶和识别转向、停车部分，考虑到这些任务对于图像的清晰度要求较高，继续使用 cam2 识别效果非常受限，同时还要始终保证巡线状态，选择用 cam0 读取的数据，提高识别效率，减少不必要的时间开销。

同时我们通过数据采集发现，cam0 采集的数据基本正常（如图15所示），但是 cam2 采集的图像存在较大畸变（如图16所示），所以我们对 cam2 图像进行矫正。矫正结果详见附录。矫正图像图和角点检测图如图17和图18所示。矫正结果如下：

```
1 RMS error: 0.388499
2 Camera matrix: [156.8597960958496, 0, 156.5480777481064;
3 0, 156.6560327205041, 127.6987415552855; 0, 0, 1]
4 Distortion coefficients :
5 [-0.3495242471365975;0.118682129385989;
6 -0.0008891403272969765;0.0008244924389640646; -0.01736317626728643]
```

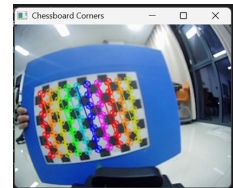


图 15: cam0 采集图像

图 16: cam2 采集图像

图 17: 矫正图像示例

图 18: 角点检测图

4.2 提升程序编译效率

由于实车调试中需要不停的调试参数，故我们的项目结构采用了基于 CMake 的多文件编译。这种模块化设计不仅提升了代码的可读性和维护性，还使得团队协作更加高效。在这个过程中，我们特别开发配置文件加载机制来管理常量配置（config.txt），在调试需要频繁修改的参数时，无需重新编译程序，极大地方便了开发和测试。这种设计允许我们在调整算法参数或运行条件时快速响应，从而提高开发效率。通过这些设计，我们确保了系统的灵活性和高性能，使得在复杂的实时图像处理场景中，系统能够高效、稳定地运行。

```
1 cmake_minimum_required(VERSION 3.10)
2 project(cpp)
3 set(CMAKE_BUILD_TYPE Release)
4 find_package(OpenCV REQUIRED)
5 set(CMAKE_CXX_FLAGS
6     "${CMAKE_CXX_FLAGS} -std=c++11 -pthread -O2 -fvisibility=hidden -s")
7 include_directories(. ${OpenCV_INCLUDE_DIRS})
8 add_executable(main main.cpp utils.cpp init.cpp tasks.cpp undistort.cpp get_binary_frame.cpp
9     transform.cpp get_lines.cpp vector_fix.cpp vector_control.cpp speed_control.cpp
10    check_cones.cpp check_directions.cpp detect_yellow_line.cpp blue_card_find.cpp check_symbol.cpp)
11 target_link_libraries(main ${OpenCV_LIBS} pigpio)
```

Listing 1: CMakeLists.txt

4.3 转向动态优化调整

在实车调试中，由于车辆在巡线过程中可能存在一定程度的倾斜，造成可能对左右转标志的误判。经分析，升级优化转向算法为动态调整规划。将上边界区域与下边界区域的三等分点修改为特定可变比例的点。经测试，0.38 的比例鲁棒性更好，判断效果最佳。

4.4 多线程设计时内存竞争读写问题

如果考虑使用多线程的方式优化任务处理效率，那么线程冲突是必然会面对的问题。

在初版线程设计中，为了尽可能大地利用 CPU 的所有逻辑处理器，我们将图像读取和图像处理分配到了不同的线程中。但是，这样的设计会发生严重的内存读写冲突。因此，应该在线程设计时尽量避免这种情况，减少异常数据的产生：

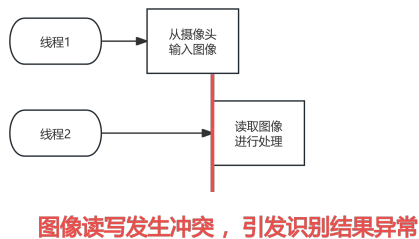


图 19: 线程冲突示意

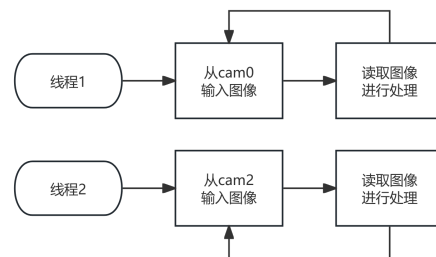


图 20: 无冲突线程设计

但是在识别斑马线时，为了减少重复的图像处理流程，采用了复用巡线的二值化结果进行二次处理。但是由于巡线在单独的线程，故采用减少读写次数的方式减少发生读写竞争的概率：

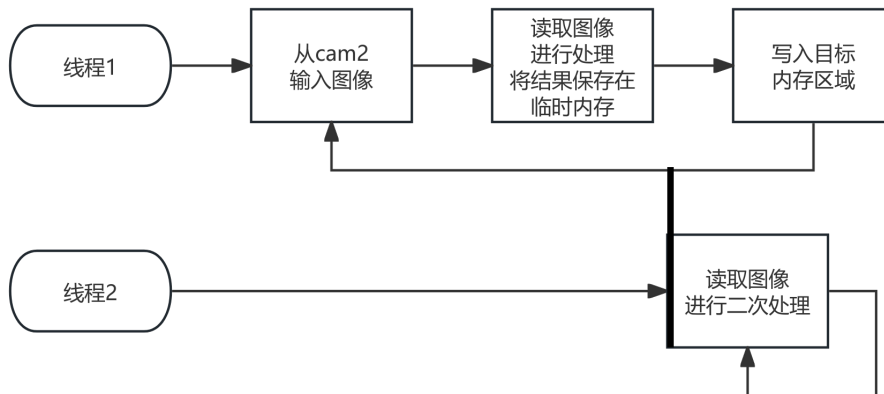


图 21: 斑马线识别线程示意

内存竞争读写的问题只在公共区域被写入的短暂瞬间才会发生，这样的设计能够大大减少内存读写竞争。

4.5 巡线方案迭代

在最初的测试中，我们简单地利用图像中白线的相对位置，使用 PID 方案调整舵机的方向。但由于即使在低速时小车的巡线震荡依然非常大，故我们考虑改进方法。

4.5.1 图像处理优化

在调试的初始阶段，我们认为是图像处理的结果不够稳定，直接导致巡线结果大幅度震荡。因此我们分析了图像中白线和蓝色地面之间的图形学特征，发现本质上二者的差异仅仅在于亮度和饱和度。故我们首先选择色彩空间变换（HSV,HLS）提取白线，但由于环境光照不稳定的原因，效果不佳。后来又尝试引入 gamma 值调整等方法也未能得到理想的效果。因此我们又尝试将图像直接进行灰度变换，并引入直方图方法稳定环境光照，在此基础上的取得了较为稳定的二值化结果：

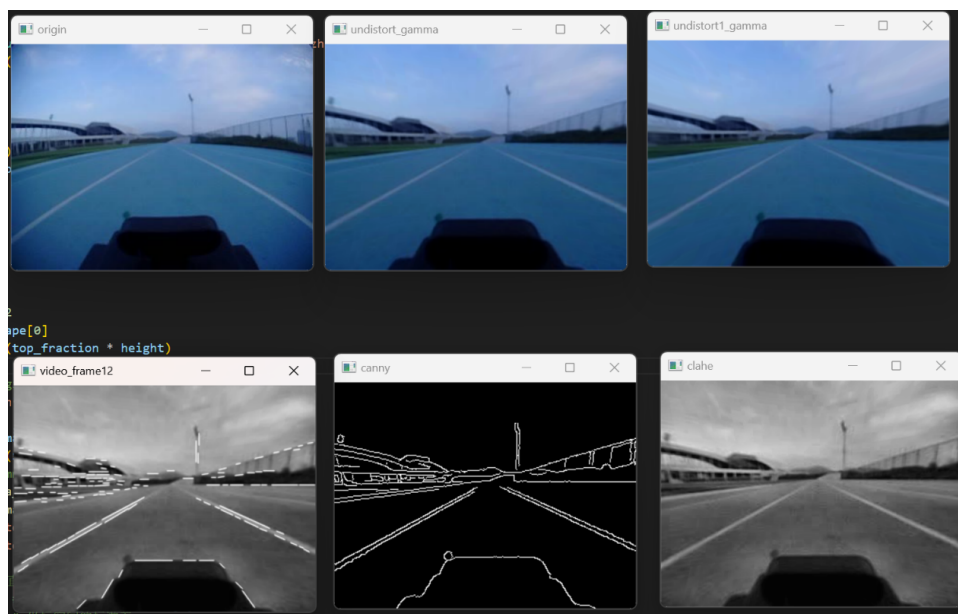


图 22: 最终二值化效果

并且，我们通过对简单的 Sobel 算子和 Canny 方法，发现 Sobel 算子的表现远不如 Canny，因此最后我们在膨胀腐蚀前使用 Canny 获取白线边缘信息。图23为二者在处理同一张图片时的测试：

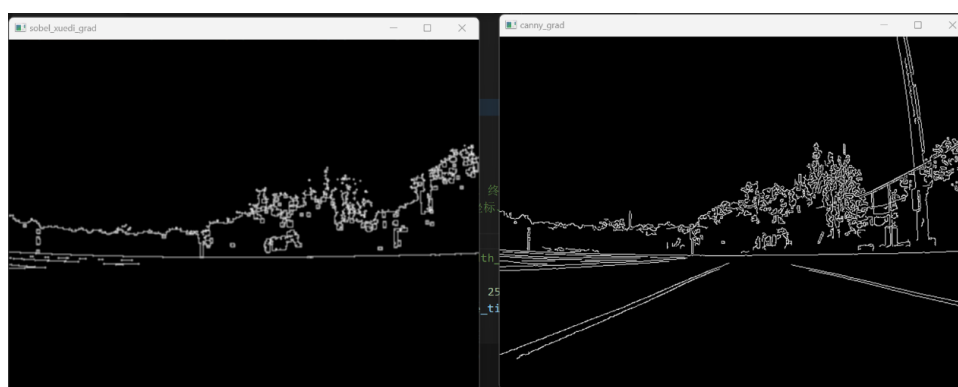


图 23: 左 Sobel 右 Canny

另外，在临近冬至，太阳高度角变小的情况下，由于操场地面纹路引起的条纹现象更为明显：



图 24: 图像中意外的噪声干扰

通过扭动 cam2 的镜头，改变焦距，降低摄像头的清晰度可以缓解这个问题。

4.5.2 引入坐标变换

通过简单的数学分析，我们发现屏幕平面空间中的直线信息与小车舵机偏转角度之间的非线性极强，直接应用 PID 方案的效果欠佳。因此，我们实地测量空间信息，通过特殊的坐标变换计算出跑道直线与小车的相对关系。图25中为测试变换坐标时的效果图，默认 output 窗口的中心位置为小车（坐标原点）。可以看到对于跑道位置的分析非常理想：

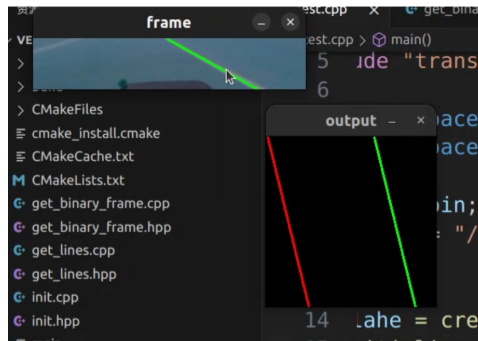


图 25: 坐标变换调试

在此基础之上应用 PID 方案进行稳定控制，使得小车的震荡范围能控制在极小的横向空间内。

4.5.3 优化控制算法

由于简单的 PID 控制方案只能控制小车在跑道中间稳定行驶，不能实现控制小车沿跑道的一侧行驶，我们引入新的控制算法，使小车速度方向向着平行于跑道的方向收敛的同时，让小车的横向位置稳定在某一点附近。但由于需要确定算法中 $|CD|$ 的值，经过实地测试有了一个大体结果。

至此，再引入 PID 方案，我们可以通过动态地调整若干参数，控制小车在跑道的特定位置进行行驶，这点优势在躲避复杂障碍时尤为明显。

4.6 开发 HSV 颜色阈值调试工具

为了便于实际调试颜色识别阈值，我们开发了一款 Python 工具，该工具能够实时提取鼠标指针所在位置的 HSV 信息。这一功能便于后续识别无法正确检测的区域，并协助调整合适的红色阈值范围。该工具的应用效果如图26所示：

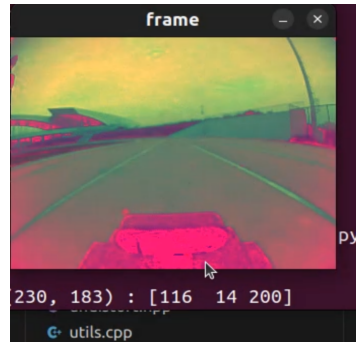


图 26: 工具使用效果

5 测试分析

数据来源 本文所有数据均由本组成员于 2024 年 7 月至 2024 年 11 月之间从智能车中采集，确保了数据的真实性和可用性。数据规模：数据包含从小车 cam0 和 cam2 读取到的在不同天气情况下的巡线、相机标定、锥桶、黄线、人行横道、蓝色挡板、转向和识别等图像，超过 8000 张图片。数据类型多样，且数据量大，可以有效反映智能车遇到的各种情况，为后续智能车可以稳定行驶提供了有力保障。

5.1 蓝色挡板识别测试

我们通过实地测试，发现用下方摄像头 cam0 读取在同一要求位置判断效果更好，蓝色区域区别更大，所以后续用 cam0 进行蓝色挡板识别判断。



图 27: cam0 对比结果

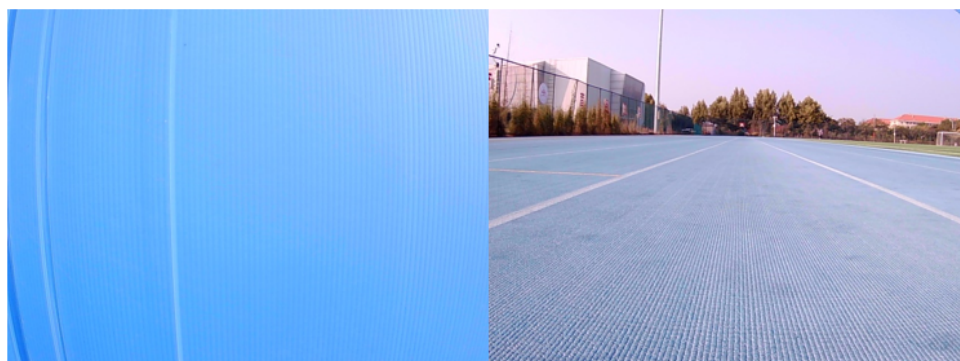


图 28: cam2 对比结果

使用数据集对算法进行测试，效果如图29所示：

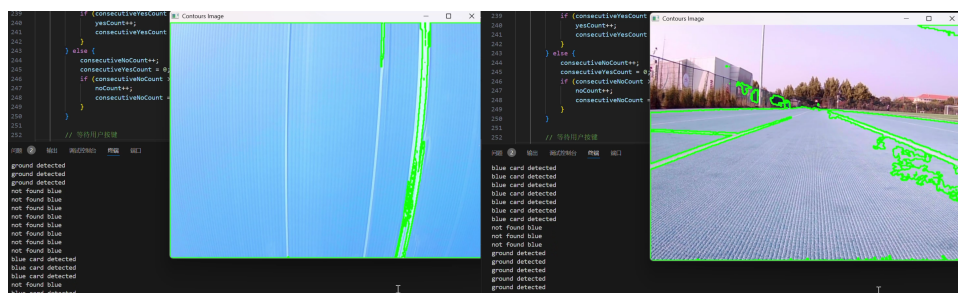


图 29: 蓝色区域识别示意图

从输出信息来看，程序准确识别了蓝色挡板和地面可以有效根据蓝色挡板是否移开控制小车行驶。

5.2 斑马线识别模块

直接复用巡线二值化图像，可以很容易得识别到斑马线，测试结果如图30所示：

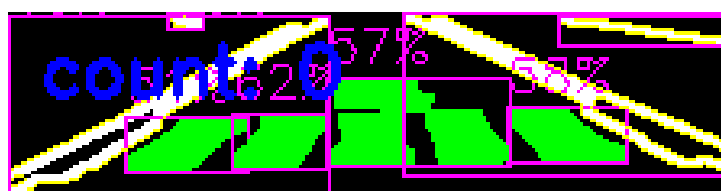


图 30: 斑马线识别示意图

5.3 转向标志识别测试

对于左右转方向的识别，本方案基于图像处理技术实现对转向标志的识别。

首先按照之前的图像处理思路，如图31所示：

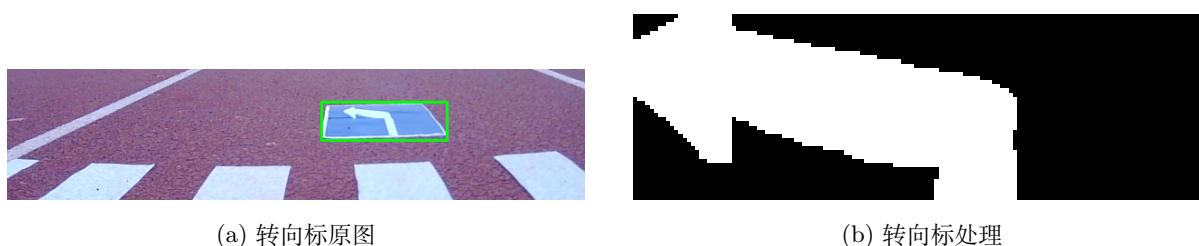


图 31: 转向标提取示意图

对于左右转标志的判定，我们采用第三章提到的坐标分析法。

5.4 锥桶识别模块

在弯道过程中，根据给定的半径阈值以及蓝色掩膜辅助观察，找寻最小外接圆同时根据宽高比的限制识别锥桶，算法对数据集中 98% 的图像做出正确的判断。实际测试时发现，对于车速不太快的情况和车速过快的情况下，都可以准确识别蓝色锥桶并且完成避障。如图32。



图 32: 弯道锥桶识别

在沿锥桶引导返回原赛道过程中，应用与弯道锥桶识别同样的算法处理，可以有效的识别锥桶，完成变道操作，如图 33所示



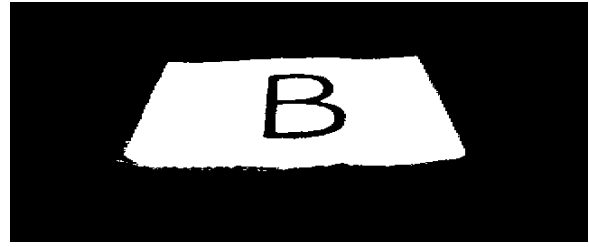
图 33: 变道锥桶识别

5.5 停车标志测试

图像处理方法与转向标志类似，图像处理结果如图34所示，可以看到图像识别十分准确：



(a) A



(b) B

图 34: 停车标志处理

6 作品总结

本方案聚焦于自主导航技术，成功完成多项任务，包括精准避障与停靠。此外，利用双摄像头和图像处理技术，方案通过巡线、障碍物识别、停车标志检测等功能，确保无人车在复杂环境下的自主导航能力。

自主导航部分采用多线程设计，优化了图像标定与矫正，提升了斑马线、转向标志、锥桶和停车位的识别精度。PID 控制算法与非线性控制方法的结合，提高了车体稳定性和响应速度。通过蓝色挡板、斑马线和转向标志的准确识别，车辆能够自动避障并精准停靠。

通过创新的硬件配置与算法优化，方案展现了远程驾驶和自主导航的高效融合，能够在多种场景下实现稳定的操作。各项技术创新如图像处理、语音播报和多任务并行处理，使系统在实际测试中表现优异，成功完成了复杂任务的自动化执行。

前期在树莓派环境配置、内容备份等部分出了很多问题，例如 python 安装包的版本不对、frp 配置失败、c++ 环境和 python 环境冲突等问题，我们一起商量实现。

在图像处理部分，我们参考书目《OpenCV 算法精解：基于 Python 与 C++》和《肖铃·学习 OpenCV4：基于 Python 的算法实战》中对解决同一类问题不同方法的优缺点以及在实际图像的测试结果进行了比对分析，找出了相对简单高效好用的方法，同时用 python 简单测试调参。

后期实地调车在更正舵机值和巡线逻辑，解决多线程冲突、时间戳错误、内存溢出、调用。本次备赛中难点主要是图像处理和速度控制。我们花了近两月时间搞定巡线部分，采集了不同环境、不同摄像头下数千张图片，经历了多次版本优化。考虑到实际地面有褪色、影子和坡度等各种实际因素的干扰，最后采用灰度直方图、裁剪等简单高效的方法实现。

我们的巡线可以在不同光照、有影子噪声等复杂情况下保持良好效果，最大可提速至小车上限速度（15000+）。

我们统筹使用了 python 和 c++，用 python 处理一些简单的事项例如立即给出图像任意点 HSV 值。我们对转向和 AB 识别部分尝试过大模型训练、轮廓识别、面积识别等方法，最后在综合考虑效率、准确率以及可行性上选择了当前的方案。

附录 Appendix

这里附上所有实现代码以及需要的配置文件，以供参考。

config.txt

```
1 kp 1
2 kd 0.01
3 hough_threshold 15
4 hough_length_limit 55
5 normal_speed 11500
6 brake_speed 7000
7 slow_speed 0.7
8 brake_sleep 1400000
9 change_track_turn_sleep 400000
10 change_track_straight_sleep 1000000
11 turn_angle 45
12 play_sound_command aplay /home/pi/test.wav
13 left_threshold 0.192
14 right_threshold 0.54
15 cross_cnt 6
```

init.hpp

```
1 #include <opencv2/opencv.hpp>
2 #include <pigpio.h>
3 #include <iostream>
4 #include <fstream>
5 #include <cstring>
6
7 #include "utils.hpp"
8 #include "transform.hpp"
9
10 double get_gamma(cv::Mat &frame);
11 void gamma_adjust(cv::Mat &frame, double gamma);
12 void get_config();
13 int init();
14 int camera_init();
15 int servo_motor_pwmInit();
```

init.cpp

```
1 #include "init.hpp"
2
3 const float servo_pwm_range = 10000,
4 servo_pwm_frequency = 50,
5 servo_pwm_duty_cycle_unlock = 720,
6 motor_pwm_range = 40000.0,
7 motor_pwm_frequency = 200.0,
8 motor_pwm_duty_cycle_unlock = 10000.0;
9
```



```
10
11 char temp[1024];
12
13 double get_gamma(cv::Mat &frame){
14     cv::Mat tmp = cv::Mat(frame, cv::Rect(int(frame.cols * 0.3), int(frame.rows * 0.6),
15                                         int(frame.cols * 0.3), int(frame.rows * 0.1)));
16     double sum = 0;
17     cvtColor(tmp, tmp, cv::COLOR_BGR2GRAY);
18     for (int i = 0; i < tmp.rows; i++)
19         for (int j = 0; j < tmp.cols; j++){
20             sum += tmp.at<uint8_t>(i, j);
21         }
22     sum /= tmp.rows * tmp.cols;
23     return (std::log(sum) - std::log(255)) / (std::log(95) - std::log(255));
24 }
25
26 void gamma_adjust(cv::Mat &frame, double gamma){
27     frame.convertTo(frame, CV_64F, 1.0/255, 0);
28     cv::pow(frame, 1 / gamma, frame);
29     frame.convertTo(frame, CV_8U, 255, 0);
30 }
31
32 void get_config(){
33     std::fstream file("config.txt", std::ios::in);
34     file >> temp >> kp >> temp >> kd
35     >> temp >> hough_threshold
36     >> temp >> hough_length_limit
37     >> temp >> normal_speed
38     >> temp >> brake_speed
39     >> temp >> slow_speed
40     >> temp >> brake_sleep
41     >> temp >> change_track_turn_sleep
42     >> temp >> change_track_straight_sleep
43     >> temp >> turn_angle
44     >> temp >> &temp[256] >> &temp[512]
45     >> temp >> left_threshold
46     >> temp >> right_threshold
47     >> temp >> cross_cnt;
48     temp[768] = ' ';
49     strcat(&temp[256], &temp[768]);
50     strcat(&temp[256], &temp[512]);
51     strncpy(play_sound_command, &temp[256], 256);
52     threshold_l = (normal_speed - 10000.0) / 1500.0 * 122.0;
53     file.close();
54     clahe = cv::createCLAHE(2.0, cv::Size(8, 8));
55 }
56
57 int init(){
58     get_config();
```

```
59     initialize_math_info();
60     return servo_motor_pwmInit() & camera_init();
61 }
62
63 int camera_init(){
64     cam0.open(0);
65     cam2.open(2);
66     if (!cam0.isOpened())
67     {
68         std::cout << "Can not open camera0!" << std::endl;
69         return 1;
70     }
71
72     if (!cam2.isOpened())
73     {
74         std::cout << "Can not open camera2!" << std::endl;
75         return 1;
76     }
77
78     cam2.set(cv::CAP_PROP_FOURCC, cv::VideoWriter::fourcc('M', 'J', 'P', 'G'));
79     cam2.set(cv::CAP_PROP_FPS, 30);
80     cam2.set(cv::CAP_PROP_FRAME_WIDTH, 320);
81     cam2.set(cv::CAP_PROP_FRAME_HEIGHT, 240);
82
83     std::cout << "Camera2 FPS: " << cam2.get(cv::CAP_PROP_FPS) << std::endl;
84     std::cout << "Camera2 Frame Width: " << cam2.get(cv::CAP_PROP_FRAME_WIDTH) << std::endl;
85     std::cout << "Camera2 Frame Height: " << cam2.get(cv::CAP_PROP_FRAME_HEIGHT) << std::endl;
86     return 0;
87 }
88
89 int servo_motor_pwmInit()
90 {
91     gpioTerminate();
92     if (gpioInitialise() < 0)
93     {
94         std::cout << "GPIO failed" << std::endl;
95         return 1;
96     }
97     else std::cout << "GPIO initialise ok" << std::endl;
98     gpioSetMode(servo_pin, PI_OUTPUT);
99     gpioSetPWmfrequency(servo_pin, servo_pwm_frequency);
100    gpioSetPwmrange(servo_pin, servo_pwm_range);
101    gpioPWM(servo_pin, servo_pwm_duty_cycle_unlock);
102
103    gpioSetMode(motor_pin, PI_OUTPUT);
104    gpioSetPWmfrequency(motor_pin, motor_pwm_frequency);
105    gpioSetPwmrange(motor_pin, motor_pwm_range);
106    gpioPWM(motor_pin, motor_pwm_duty_cycle_unlock);
107 }
```

```

108     gpioSetMode(cradlex_pin, PI_OUTPUT);
109     gpioSetPWmfrequency(cradlex_pin, servo_pwm_frequency);
110     gpioSetPWMrange(cradlex_pin, servo_pwm_range);
111     gpioPWM(cradlex_pin, servo_pwm_duty_cycle_unlock);
112
113     gpioSetMode(cradley_pin, PI_OUTPUT);
114     gpioSetPWmfrequency(cradley_pin, servo_pwm_frequency);
115     gpioSetPWMrange(cradley_pin, servo_pwm_range);
116     gpioPWM(cradley_pin, servo_pwm_duty_cycle_unlock - 5);
117     return 0;
118 }

```

blue_card_find.hpp

```

1 #include <opencv2/opencv.hpp>
2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5
6 bool blue_card_find(const cv::Mat& frame);

```

blue_card_find.cpp

```

1 #include "blue_card_find.hpp"
2
3 cv::Scalar lowerBlue(100, 43, 46);
4 cv::Scalar upperBlue(124, 255, 255);
5 const double minNonBlueAreaRatio = 0.18; // 非蓝色区域最小占屏幕面积比例
6 const double maxNonBlueAreaRatio = 0.05; // 非蓝色区域最大占屏幕面积比例
7
8 bool blue_card_find(const cv::Mat& frame) {
9     cv::Size imageSize = frame.size();
10
11     cv::Mat hsv, mask;
12     cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
13     cv::inRange(hsv, lowerBlue, upperBlue, mask);
14
15     cv::erode(mask, mask, cv::Mat(), cv::Point(-1, -1), 2);
16     cv::GaussianBlur(mask, mask, cv::Size(3, 3), 0);
17
18     // 计算非蓝色区域的比例
19     double nonBluePixels = mask.total() - countNonZero(mask);
20     double totalPixels = mask.rows * mask.cols;
21     double nonBlueRatio = nonBluePixels / totalPixels;
22     std::cout << nonBlueRatio << std::endl;
23     if (nonBlueRatio > minNonBlueAreaRatio) {
24         // std::cout << "ground detected" << std::endl;
25         return true;

```

```
26     }
27     // std::cout << "blue card detected" << std::endl;
28     return false;
29
30 }
```

check_cones.hpp

```
1 #include <vector>
2 #include <opencv2/opencv.hpp>
3
4 #include "utils.hpp"
5
6 void cones_image_process(const cv::Mat& frame, int& zeroCount);
```

check_cones.cpp

```
1 #include "check_cones.hpp"
2
3
4 cv::Scalar lowerBound1(0, 100, 50);
5 cv::Scalar upperBound1(10, 255, 255);
6 cv::Scalar lowerBound2(160, 100, 50);
7 cv::Scalar upperBound2(180, 255, 255);
8 std::vector<std::vector<cv::Point>> cnts;
9 cv::Size imageSize;
10 float minRadius = 1.0f;
11
12 void cones_image_process(const cv::Mat& lowerRegion, int& check_count) {
13     cv::Mat hsv, mask1, mask2, mask;
14     cv::cvtColor(lowerRegion, hsv, cv::COLOR_BGR2HSV);
15     cv::inRange(hsv, lowerBound1, upperBound1, mask1);
16     cv::inRange(hsv, lowerBound2, upperBound2, mask2);
17
18     // 合并两个掩膜
19     mask = mask1 | mask2;
20     //mask = mask2;
21     // 腐蚀和高斯模糊
22     cv::erode(mask, mask, cv::Mat(), cv::Point(-1, -1), 2);
23     cv::GaussianBlur(mask, mask, cv::Size(3, 3), 0);
24
25     // 查找轮廓
26     std::vector<std::vector<cv::Point>> cnts;
27     //cv::imshow("img", mask);
28     cv::findContours(mask, cnts, cv::RETR_EXTERNAL, cv::CHAIN_APPROX_SIMPLE);
29
30     for (const auto& contour : cnts) {
31         float radius;
```

```
32         cv::Point2f center;
33         cv::minEnclosingCircle(contour, center, radius); // 计算最小外接圆
34
35         // 检查半径
36         std::cout << radius << std::endl;
37         if (radius > minRadius && radius < 55.0f) {
38             check_count++;
39             return;
40         }
41     }
42     }
43     check_count = 0;
44 }
```

check_directions.hpp

```
1 #include <opencv2/opencv.hpp>
2 #include <iostream>
3
4 // 处理图像的函数
5 char check_directions(const cv::Mat& inputImage, cv::Mat& redMask, cv::Mat& contoursImage);
```

check_directions.cpp

```
1 #include "check_directions.hpp"
2
3 using namespace std;
4 using namespace cv;
5
6 // 处理图像的函数
7 char check_directions(const cv::Mat& inputImage, cv::Mat& redMask, cv::Mat& contoursImage) {
8     Mat lowerQuarter = inputImage;
9     Mat hsvImage;
10    cvtColor(lowerQuarter, hsvImage, COLOR_BGR2HSV);
11
12    // 设置红色的HSV范围
13    Scalar lowerRed1(0, 50, 50);
14    Scalar upperRed1(10, 255, 255);
15    Scalar lowerRed2(160, 50, 50);
16    Scalar upperRed2(180, 255, 255);
17
18    // 创建掩膜
19    Mat mask1, mask2;
20    inRange(hsvImage, lowerRed1, upperRed1, mask1);
21    inRange(hsvImage, lowerRed2, upperRed2, mask2);
22    redMask = mask1 | mask2;
23
24    // 进行轮廓检测
```

```
25     vector<vector<Point>> contours;
26     vector<Vec4i> hierarchy;
27     findContours(redMask, contours, hierarchy, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
28
29     // 创建轮廓图像并绘制轮廓
30     contoursImage = lowerQuarter.clone();
31     drawContours(contoursImage, contours, -1, Scalar(0, 255, 0), 1);
32     //imshow("out", contoursImage);
33     //waitKey(0);
34     double leftArea = 0;
35     double rightArea = 0;
36     double leftWidth = 0; // 左边轮廓的宽度
37     double rightWidth = 0; // 右边轮廓的宽度
38
39     for (const auto& contour : contours) {
40         if (contour.size() < 80) continue; // 忽略小轮廓
41
42         // 获取轮廓的最左、最右、上和下边界点
43         Point leftEdge = contour[0];
44         Point rightEdge = contour[0];
45         Point topEdge = contour[0];
46         Point bottomEdge = contour[0];
47
48         for (const auto& point : contour) {
49             if (point.x < leftEdge.x) {
50                 leftEdge = point;
51             }
52             if (point.x > rightEdge.x) {
53                 rightEdge = point;
54             }
55             if (point.y < topEdge.y) {
56                 topEdge = point;
57             }
58             if (point.y > bottomEdge.y) {
59                 bottomEdge = point;
60             }
61         }
62
63         // 计算轮廓的高度
64         double height = bottomEdge.y - topEdge.y;
65
66         // 计算与 topEdge.y + width / 2 的平行线的y值
67         double lineY = topEdge.y + height * 0.4;
68
69         // 计算与该平行线的交点
70         std::vector<int> intersectionXLineY;
71         for (size_t i = 0; i < contour.size(); ++i) {
72             Point p1 = contour[i];
73             Point p2 = contour[(i + 1) % contour.size()]; // 获取下一个点
```

```

74
75         // 检查是否与指定的平行线相交
76         if ((p1.y < lineY && p2.y >= lineY) || (p1.y >= lineY && p2.y < lineY)) {
77             // 线性插值计算交点
78             double t = (lineY - p1.y) / (p2.y - p1.y);
79             int intersectX = static_cast<int>(p1.x + t * (p2.x - p1.x));
80             intersectionXLineY.push_back(intersectX);
81         }
82     }
83
84     // 如果没有与指定平行线的交点，则跳过此轮廓
85     if (intersectionXLineY.empty()) {
86         continue;
87     }
88
89     // 计算所有交点的横坐标平均值
90     double averageIntersectX = 0.0;
91     for (int x : intersectionXLineY) {
92         averageIntersectX += x;
93     }
94     averageIntersectX /= intersectionXLineY.size();
95
96     // 获取轮廓的宽度
97     double width = rightEdge.x - leftEdge.x;
98
99     // 输出边界点和高度信息
100    /*std::cout << "左边界: (" << leftEdge.x << ", " << leftEdge.y << "), "
101    << "右边界: (" << rightEdge.x << ", " << rightEdge.y << "), "
102    << "上边界: (" << topEdge.x << ", " << topEdge.y << "), "
103    << "下边界: (" << bottomEdge.x << ", " << bottomEdge.y << "), "
104    << "高度: " << height << "\n";*/
105
106    // 判断方向
107    if (averageIntersectX < (leftEdge.x + width / 2)) {
108        return 0;
109    } else {
110        return 1;
111    }
112 }
113
114 // 默认返回
115 return -1;
116 }

```

check_symbol.hpp

```

1 #include <opencv2/opencv.hpp>
2 #include <iostream>
3 #include <vector>

```

```
4  
5 int check_symbol(cv::Mat image);
```

check_symbol.cpp

```
1 #include "check_symbol.hpp"  
2  
3 using namespace cv;  
4 using namespace std;  
5  
6 int check_symbol(cv::Mat image) {  
7     // 获取图像的高度和宽度  
8     int height = image.rows;  
9     int width = image.cols;  
10  
11     // 裁剪图像的下半部分  
12     Mat cropped_image = image(Rect(0, height / 2, width, height / 2));  
13  
14     // 将目标颜色从 RGB 转换到 HSV  
15     Vec3b color_rgb(203, 115, 153); // 输入 RGB 颜色  
16     Mat color_hsv;  
17     cvtColor(Mat(1, 1, CV_8UC3, color_rgb), color_hsv, COLOR_BGR2HSV);  
18     Vec3b target_hsv = color_hsv.at<Vec3b>(0, 0);  
19     cout << "HSV_now: (" << (int)target_hsv[0] << ", " << (int)target_hsv[1] << ", " << (int)  
        target_hsv[2] << ")" << endl;  
20  
21     // 定义目标颜色的 HSV 范围  
22     Scalar lower_bound(150, 30, 40); // 下限  
23     Scalar upper_bound(180, 250, 255); // 上限  
24  
25     // 转换裁剪后的图像到 HSV  
26     Mat hsv_image;  
27     cvtColor(cropped_image, hsv_image, COLOR_BGR2HSV);  
28  
29     // 创建颜色区域的掩码  
30     Mat mask;  
31     inRange(hsv_image, lower_bound, upper_bound, mask);  
32  
33     // 查找连通区域 (轮廓)  
34     vector<vector<Point>> contours;  
35     vector<Vec4i> hierarchy;  
36     findContours(mask, contours, hierarchy, RETR_CCOMP, CHAIN_APPROX_SIMPLE);  
37  
38     // 初始化最大连通区域的面积和外轮廓的面积  
39     double max_connected_area = 0;  
40     int max_contour_index = -1;  
41     vector<Point> max_c;  
42  
43     // 遍历所有轮廓, 找到最大连通区域
```



```

44     for (size_t i = 0; i < contours.size(); i++) {
45         double area = contourArea(contours[i]);
46         if (area > max_connected_area) {
47             max_connected_area = area;
48             max_contour_index = i;
49             max_c = contours[i];
50         }
51     }
52
53     // 创建一个新的空掩码用于绘制最大连通区域
54     // Mat max_area_mask = Mat::zeros(mask.size(), CV_8UC1);
55     // drawContours(max_area_mask, contours, max_contour_index, Scalar(255), FILLED);
56
57     int count_inside = 0;
58
59     for (size_t i = 0; i < contours.size(); i++) {
60         // Mat mask_for_contour = Mat::zeros(mask.size(), CV_8UC1);
61         // drawContours(mask_for_contour, contours, i, Scalar(255), FILLED);
62         double area = contourArea(contours[i]);
63
64         try {
65             Moments moments = cv::moments(contours[i]);
66             int cx = moments.m00 != 0 ? int(moments.m10 / moments.m00) : 0;
67             int cy = moments.m00 != 0 ? int(moments.m01 / moments.m00) : 0;
68
69             int inside = pointPolygonTest(max_c, Point(cx, cy), false);
70
71             if (50 < area && area < max_connected_area - 1 && inside > 0) {
72                 count_inside++;
73             }
74         } catch (const std::exception& e) {
75             cout << "Error calculating moments for contour " << i + 1 << ": " << e.what
76                 << endl;
77         }
78     }
79
80     cout << count_inside << endl;
81     if (count_inside == 2) {
82         cout << "a" << endl;
83         return 0;
84     } else if (count_inside == 3) {
85         cout << "b" << endl;
86         return 1;
87     } else {
88         cout << "???" << endl;
89         return -1;
90     }

```

detect_yellow_line.hpp

```
1 #include <opencv2/opencv.hpp>
2 #include <vector>
3
4 using namespace cv;
5 using namespace std;
6
7 bool detect_yellow_line(cv::Mat img);
```

detect_yellow_line.cpp

```
1 #include "detect_yellow_line.hpp"
2
3 using namespace cv;
4 using namespace std;
5
6 bool detect_yellow_line(Mat img) { //red line, actually
7     Mat cropped_image, mask;
8     // Scalar lowerb = Scalar(26, 43, 46); // 黄色的HSV下界
9     // Scalar upperb = Scalar(34, 255, 255); // 黄色的HSV上界
10    Scalar lowerb = Scalar(155, 30, 40); // 黄色的HSV下界
11    Scalar upperb = Scalar(180, 250, 255); // 黄色的HSV上界
12
13    int height = img.rows;
14    int width = img.cols;
15    int half_height = int(height / 2);
16
17    // 裁剪图像下半部分
18    cropped_image = img(Rect(0, half_height, width, half_height)).clone();
19    cvtColor(cropped_image, cropped_image, COLOR_BGR2HSV);
20    inRange(cropped_image, lowerb, upperb, mask); // 生成黄色部分的掩膜
21    morphologyEx(mask, mask, MORPH_OPEN, getStructuringElement(MORPH_RECT, Size(3, 3)));
22
23    // 检测轮廓
24    vector<vector<Point>> contours;
25    findContours(mask, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
26    bool yellow_detected = false;
27
28    for (size_t i = 0; i < contours.size(); i++) {
29        double area = contourArea(contours[i]);
30        cout << "area: " << area << endl;
31        if (area > 50) { // 根据实际情况调整面积阈值
32            Point2f center;
33            float radius;
34            minEnclosingCircle(contours[i], center, radius);
35            cout << "r: " << radius << endl;
36            if (radius > 10) { // 根据实际情况调整半径阈值
37                yellow_detected = true;
```

```

38         }
39     }
40 }
41
42     return yellow_detected;
43 }

```

get_binary_frame.hpp

```

1 #include <opencv2/opencv.hpp>
2
3 #include "utils.hpp"
4
5 void get_binary_frame_track(cv::Mat &input_frame, cv::Mat &bin_frame);

```

get_binary_frame.cpp

```

1 #include "get_binary_frame.hpp"
2
3 const cv::Size const_size(5, 5);
4 cv::Mat kernel = cv::getStructuringElement(cv::MORPH_RECT, cv::Size(3, 3));
5 cv::Mat bin_temp;
6
7 void get_binary_frame_track(cv::Mat &input_frame, cv::Mat &bin_frame){
8     cv::cvtColor(input_frame, bin_temp, cv::COLOR_BGR2GRAY);
9     clahe->apply(bin_temp, bin_temp);
10    cv::GaussianBlur(bin_temp, bin_temp, const_size, 0.52);
11    cv::Canny(bin_temp, bin_temp, 50, 140);
12    cv::dilate(bin_temp, bin_frame, kernel, cv::Point(-1, -1), 1);
13 }

```

get_lines.hpp

```

1 #include <vector>
2 #include "transform.hpp"
3 #include "utils.hpp"
4
5 extern const double track_width;
6
7 //extern void (*map_modes[4])(const Line &, const Line &, Line &, Line &);
8
9 int get_lines(const cv::Mat &bin_frame, Line &l_line, Line &r_line);
10 void map_lines(const Line &l_s_line, const Line &r_s_line,
11 Line &l_r_line, Line &r_r_line, int mode);
12 void map_mode0(const Line &l_s_line, const Line &r_s_line,
13 Line &l_r_line, Line &r_r_line);
14 void map_mode1(const Line &l_s_line, const Line &r_s_line,
15 Line &l_r_line, Line &r_r_line);

```

```

16 void map_mode2(const Line &l_s_line, const Line &r_s_line,
17 Line &l_r_line, Line &r_r_line);
18 void map_mode3(const Line &l_s_line, const Line &r_s_line,
19 Line &l_r_line, Line &r_r_line);

```

get_lines.cpp

```

1  #include "get_lines.hpp"
2
3  const double track_width = 137.273;
4
5  void (*map_modes[4])(const Line &, const Line &, Line &, Line &) = {
6      &map_mode0, &map_mode1, &map_mode2, &map_mode3
7  };
8  std::vector<cv::Vec4i> tmp;
9  int get_lines(const cv::Mat& bin_frame, Line &l_line, Line &r_line) {
10     tmp.clear();
11     double k, l_b = 0, r_b = 0; cv::Point2d temp, vec, l_line_vec, r_line_vec;
12     double l_line_cnt, r_line_cnt; l_line_cnt = r_line_cnt = 0;
13     r_line_vec.x = r_line_vec.y = l_line_vec.x = l_line_vec.y = 0;
14     int ret = 0;
15
16     HoughLinesP(bin_frame, tmp, 1, M_PI / 180, hough_threshold, hough_length_limit);
17     for (auto i : tmp) {
18         if (abs(i[0] - i[2]) < 3 || abs(i[1] - i[3]) < 3) continue;
19
20         k = double(i[1] - i[3]) / double(i[0] - i[2]);
21         temp = cv::Point2d(double(i[0]), double(i[1]));
22         if (abs(k) > 0.25) {
23             vec = cv::Point2d(i[0] - i[2], i[1] - i[3]);
24             if (k < 0){
25                 l_line_vec += vec / vec.ddot(vec);
26                 l_b += (temp.y - (vec.y / vec.x) * temp.x) * sqrt(vec.ddot(vec));
27                 l_line_cnt += sqrt(vec.ddot(vec));
28             }
29             else{
30                 r_line_vec += vec;
31                 r_b += (temp.y - (vec.y / vec.x) * temp.x) * sqrt(vec.ddot(vec));
32                 r_line_cnt += sqrt(vec.ddot(vec));
33             }
34         }
35     }
36
37     if (l_line_cnt > 1) {
38         l_line.m = l_line_vec.y / l_line_vec.x;
39         l_line.b = l_b / l_line_cnt;
40         ret |= 1;
41     }
42

```

```

43     if (r_line_cnt > 1) {
44         r_line.m = r_line_vec.y / r_line_vec.x;
45         r_line.b = r_b / r_line_cnt;
46         ret |= 2;
47     }
48     return ret;
49 }
50
51 void map_lines(const Line &l_s_line, const Line &r_s_line,
52 Line &l_r_line, Line &r_r_line, int mode) {
53     (map_modes[mode])(l_s_line, r_s_line, l_r_line, r_r_line);
54 }
55
56 void map_mode0(const Line &, const Line &, Line &, Line &){
57     return;
58 }
59
60 void map_mode1(const Line &l_s_line, const Line &r_s_line,
61 Line &l_r_line, Line &r_r_line) {
62     cv::Point2d *start, *end, vec;
63     double k = l_s_line.m, b = l_s_line.b;
64     start = &pos_map[5][int((5 - b) / k)];
65     if (55 < k * 5 + b){
66         end = &pos_map[55][int((55 - b) / k)];
67     }
68     else {
69         end = &pos_map[int(5 * k + b)][5];
70     }
71     vec = *end - *start;
72     l_r_line.m = vec.x / vec.y;
73     l_r_line.b = end->x - end->y * l_r_line.m;
74     r_r_line.m = l_r_line.m;
75     r_r_line.b = l_r_line.b + track_width / cos(atanf64(r_r_line.m));
76 }
77 }
78
79 void map_mode2(const Line &l_s_line, const Line &r_s_line,
80 Line &l_r_line, Line &r_r_line) {
81     cv::Point2d *start, *end, vec;
82     double k = r_s_line.m, b = r_s_line.b;
83     start = &pos_map[5][int((5 - b) / k)];
84     if (55 < k * 305 + b) {
85         end = &pos_map[55][int((55 - b) / k)];
86     }
87     else {
88         end = &pos_map[int(305 * k + b)][305];
89     }
90     vec = *end - *start;
91     l_r_line.m = r_r_line.m = vec.x / vec.y;

```

```

92     r_r_line.b = end->x - end->y * r_r_line.m;
93     l_r_line.b = r_r_line.b - track_width / cos(atanf64(l_r_line.m));
94
95 }
96
97 void map_mode3(const Line &l_s_line, const Line &r_s_line,
98 Line &l_r_line, Line &r_r_line){
99     cv::Point2d *start, *end, vec, mid;
100     double k = l_s_line.m, b = l_s_line.b;
101     start = &pos_map[5][int((5 - b) / k)];
102     if (55 < k * 5 + b){
103         end = &pos_map[55][int((55 - b) / k)];
104     }
105     else {
106         end = &pos_map[int(5 * k + b)][5];
107     }
108     vec = *end - *start;
109     mid = (*end + *start) * 0.5;
110
111     k = r_s_line.m, b = r_s_line.b;
112     start = &pos_map[5][int((5 - b) / k)];
113     if (55 < k * 305 + b) {
114         end = &pos_map[55][int((55 - b) / k)];
115     }
116     else {
117         end = &pos_map[int(305 * k + b)][305];
118     }
119
120     vec += *end - *start;
121     r_r_line.m = l_r_line.m = vec.x / vec.y;
122     l_r_line.b = mid.x - l_r_line.m * mid.y;
123     mid = (*end + *start) * 0.5;
124     r_r_line.b = mid.x - r_r_line.m * mid.y;
125
126 }

```

main.cpp

```

1 #include <csignal>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 #include "utils.hpp"
6 #include "init.hpp"
7 #include "undistort.hpp"
8 #include "speed_control.hpp"
9 #include "vector_control.hpp"
10 #include "vector_fix.hpp"
11 #include "get_binary_frame.hpp"

```

```

12 #include "get_lines.hpp"
13 #include "tasks.hpp"
14
15 pthread_t trace_thread, task_thread;
16 int *thread_ret;
17 int main(){
18     if (init()) {
19         // 初始化出错, 退出并返回-1
20         std::cout << "Init error (-1)" << std::endl;
21         return -1;
22     }
23     // 检查蓝色挡板, 阻塞, 直到蓝色挡板移除
24     if (check_blue_plain()) {
25         // 蓝色挡板识别发生错误, 退出并返回-9
26         std::cout << "Check blue plain error (-9)" << std::endl;
27         return -9;
28     }
29     std::cout << "蓝色挡板移除" << std::endl;
30     if (pthread_create(&trace_thread, nullptr, trace, nullptr)) {
31         std::cout << "trace error (-10)" << std::endl;
32         return -10;
33     }
34     usleep(500000);
35     target_threshold = 0.41; // 0.41
36     kick_start();
37     set_speed(14000);
38     //usleep(2500000);
39     usleep(22500000);
40     set_speed(12400);
41     usleep(4000000);
42     //set_speed(11450);
43     //set_speed(12500);
44     //usleep(4000000);
45     set_speed(11400);
46     //for (int i = 0; i < 10; i++) cam0.read(input_frame0);
47     usleep(500000);
48
49     if (pthread_create(&task_thread, nullptr, check_crosswalk, nullptr)) {
50         // 启动斑马线检测线程出错, 退出并返回-2
51         std::cout << "Start error (-2)" << std::endl;
52         return -2;
53     }
54
55     // 阻塞, 等待check_crosswalk退出 (即识别到斑马线)
56     pthread_join(task_thread, (void **)&thread_ret);
57     if (*thread_ret) {
58         // 线程运行时出错, 退出并返回-3
59         std::cout << "Check crosswalk error (-3)" << std::endl;
60         return -3;

```

```
61     }
62     //刹车, 识别箭头标志, 准备变道
63     brake();
64     delete thread_ret;
65     if (pthread_create(&task_thread, nullptr, check_direction, nullptr)) {
66         // 启动识别标志线程失败, 退出并返回-4
67         std::cout << "Change track error (-4)" << std::endl;
68         return -4;
69     }
70     // 分离标志识别线程
71     pthread_detach(task_thread);
72     // 杀死巡线线程
73     //pthread_kill(trace_thread, 0);
74     auto_mode = false;
75     // 语音播报
76     system(play_sound_command);
77     // 等待5秒, 假设识别标志线程已经在等待后结束
78     usleep(1000000);
79     // 根据标志识别线程的结果进行变道
80
81     change_track();
82     auto_mode = true;
83     usleep(1000000);
84     // usleep(2000000); return 0;
85     // 启动锥桶检测线程
86     if (pthread_create(&task_thread, nullptr, check_cones, nullptr)) {
87         // 启动锥桶检测失败, 退出并返回-5
88         std::cout << "check cones error (-5)" << std::endl;
89         return -5;
90     }
91     set_speed(12800);
92
93     // 阻塞并等待锥桶检测线程结束
94     pthread_join(task_thread, (void **)&thread_ret);
95
96     if (*thread_ret){
97         // 锥桶检测运行时异常, 退出并返回-7
98         std::cout << "check cones runtime error (-7)" << std::endl;
99         return -7;
100    }
101
102    // 启动黄线检测线程
103    if (pthread_create(&task_thread, nullptr, check_yellowline, nullptr)){
104        // 启动黄线检测线程失败, 退出并返回-6
105        std::cout << "Check yellowline error (-6)" << std::endl;
106        return -6;
107    }
108    set_speed(14000);
109    usleep(9000000);
```



```
110     set_speed(9500);
111     usleep(300000);
112     set_speed(11700);
113     // 阻塞并等待黄线检测线程结束
114     pthread_join(task_thread, (void **)&thread_ret);
115
116     if (*thread_ret){
117         // 黄线检测运行时错误, 退出并返回-8
118         std::cout << "Check yellowline runtime error (-8)" << std::endl;
119         return -8;
120     }
121     usleep(100000);
122     // 减速准备停车
123     slow_down();
124     // 识别标志, 根据标志位置调整停车区域
125     // 1: right; 0: left
126
127     if (get_symbol()) target_threshold = right_threshold;
128     else target_threshold = 0.16;
129
130
131     // 通过巡线找到停车位置
132     usleep(brake_sleep);
133     // 停车
134     brake();
135
136     // 回收巡线线程, 结束程序
137     pthread_kill(trace_thread, 0);
138     std::cout << "Finished." << std::endl;
139     return 0;
140 }
```

speed_control.hpp

```
1 #include <pigpio.h>
2 #include <unistd.h>
3
4 #include "utils.hpp"
5
6 void brake();
7 void kick_start();
8 void slow_down();
9 void set_speed(int speed);
```

speed_control.cpp

```
1 #include "speed_control.hpp"
2
```

```
3 const int stop_speed = 10000;
4
5 void brake(){
6     gpioPWM(motor_pin, brake_speed);
7     usleep(750000);
8     gpioPWM(motor_pin, stop_speed);
9 }
10
11 void kick_start(){
12     gpioPWM(motor_pin, normal_speed + 500);
13     usleep(16667);
14     gpioPWM(motor_pin, normal_speed);
15 }
16
17 void slow_down(){
18     gpioPWM(motor_pin, int((normal_speed - stop_speed) * slow_speed + 10000));
19 }
20
21 void set_speed(int speed){
22     gpioPWM(motor_pin, speed);
23 }
```

tasks.hpp

```
1 #include <pthread.h>
2 #include <unistd.h>
3 #include <opencv2/opencv.hpp>
4 #include <limits>
5 #include <csignal>
6
7 #include "utils.hpp"
8 #include "undistort.hpp"
9 #include "get_binary_frame.hpp"
10 #include "get_lines.hpp"
11 #include "vector_fix.hpp"
12 #include "vector_control.hpp"
13 #include "transform.hpp"
14 #include "speed_control.hpp"
15 #include "check_cones.hpp"
16 #include "check_directions.hpp"
17 #include "detect_yellow_line.hpp"
18 #include "blue_card_find.hpp"
19 #include "check_symbol.hpp"
20
21 extern bool check_result;
22
23 void *trace(void *);
24 void *check_crosswalk(void *);
25 void change_track();
```

```

26 void *check_direction(void *);
27 void *check_cones(void *);
28 void *check_yellowline(void *);
29 bool get_symbol();
30 int check_blue_plain();

```

tasks.cpp

```

1  #include "tasks.hpp"
2
3  pthread_mutex_t mmutex = PTHREAD_MUTEX_INITIALIZER;
4  bool check_result = true; // false: left; true: right
5
6  void *trace(void*) {
7      int *ret = new int; *ret = -1;
8      pthread_mutex_lock(&mutex);
9      pthread_detach(pthread_self());
10     if (!cam2.read(input_frame2) || input_frame2.empty()) {
11         std::cout << "Failed to read image from Camera2!" << std::endl;
12         pthread_exit((void *)ret);
13     }
14     pthread_mutex_unlock(&mutex);
15     std::cout << "running task1" << std::endl;
16
17     do {
18         cam2.read(input_frame2);
19         undistort(input_frame2);
20         input_frame2 = cv::Mat(input_frame2, cv::Rect(0, 120, 320, 60));
21         get_binary_frame_track(input_frame2, binary_frame2);
22         hough_result = get_lines(binary_frame2, left_screen_line, right_screen_line);
23         map_lines(left_screen_line, right_screen_line,
24             left_real_line, right_real_line, hough_result);
25         if (auto_mode) servo_control(get_error_angle(left_real_line, right_real_line,
26             target_threshold));
27         // double error = get_error_angle(left_real_line, right_real_line, 0.4);
28         // std::cout << "error: " << error << std::endl;
29         // servo_control(error);
30         // std::cout << "time usage: " << double(clock() - start) / CLOCKS_PER_SEC * 1000 <<
31             "ms" << std::endl;
32     } while (cam2.read(input_frame2) && !input_frame2.empty());
33     pthread_exit((void *)ret);
34 }
35 void *check_crosswalk(void*){
36     int cnt = 0, *ret = new int; *ret = -1;
37     std::vector<std::vector<cv::Point> > contours;
38     while (!binary_frame2.empty()){
39         findContours(binary_frame2, contours, cv::RETR_EXTERNAL, cv::CHAIN_APPROX_NONE);

```

```
40         cnt = 0;
41         for (std::vector<cv::Point> vec : contours){
42             if (vec.size() > 70) cnt++;
43             else cnt = 0;
44         }
45         //std::cout << cnt << std::endl;
46         if (cnt >= cross_cnt) {
47             *ret = 0;
48             //usleep(10000);
49             break;
50         }
51     }
52     pthread_exit((void *)ret);
53 }
54
55 void change_track(){
56     if (check_result) turn_angle *= -1;
57     kick_start();
58     set_speed(11700);
59     usleep(500000);
60     servo_control(-turn_angle);
61     usleep(change_track_turn_sleep);
62     servo_control(0);
63     usleep(change_track_straight_sleep);
64 }
65
66 void *check_cones(void *){
67     int *ret = new int, check_count = 0, step = 0; *ret = -1;
68     double targets[4] = {left_threshold, right_threshold, left_threshold, 0.41};
69     target_threshold = targets[0];
70     for (int i = 0; i < 4; i++) cam0.read(input_frame0);
71     while (!input_frame0.empty() && cam0.read(input_frame0)){
72         cones_image_process(cv::Mat(input_frame0, cv::Rect(0, 260, 640, 220)), check_count);
73         std::cout << step << ' ' << check_count << std::endl;
74         if (check_count == 1) {
75             usleep(300000);
76             for (int i = 0; i < 4; i++) cam0.read(input_frame0);
77             std::cout << step << std::endl;
78             target_threshold = targets[++step];
79             check_count = 0;
80             if (step == 3) {
81                 usleep(100000);
82                 *ret = 0;
83                 break;
84             }
85         }
86         cam0.read(input_frame0);
87     }
88     pthread_exit((void *)ret);
```

```
89     return ret;
90 }
91
92 void *check_direction(void *){
93     int *ret = new int, cnt = 0; *ret = -1;
94     check_result = false;
95     *ret = 0;
96     pthread_exit((void *)ret);
97     return ret;
98     cam0.read(input_frame0);
99     cv::Mat frame, redMask, contoursImage;
100     while (cnt <= 5 and cnt >= -5 and !input_frame0.empty()){
101         cam0.read(input_frame0);
102         frame = cv::Mat(input_frame0, cv::Rect(0, 200, 640, 280));
103         switch (check_directions(frame, redMask, contoursImage)){
104             case 0:
105                 cnt -= 1;
106                 break;
107             case 1:
108                 cnt += 1;
109                 break;
110             default:
111                 break;
112         }
113         std::cout << cnt << std::endl;
114     }
115     check_result = (cnt == 6);
116     *ret = 0;
117     pthread_exit((void *)ret);
118 }
119
120 void *check_yellowline(void *){
121     int *ret = new int; *ret = -1;
122     for (int i = 0; i < 4; i++) cam0.read(input_frame0);
123     while (!input_frame0.empty()) {
124         if (detect_yellow_line(input_frame0)){
125             *ret = 0;
126             break;
127         }
128         cam0.read(input_frame0);
129     }
130     pthread_exit((void *)ret);
131 }
132
133
134
135 bool get_symbol() {
136     int right_turn_count = 0;
137     int left_turn_count = 0;
```

```

138     cv::Mat redMask, contoursImage;
139     while (!input_frame0.empty()) {
140         int result = check_symbol(input_frame0);
141         // cv::imshow("output",input_frame0);
142         // std::cout << "result Here" << std::endl;
143         if (result==1) {
144             right_turn_count++;
145             left_turn_count = 0; // 重置右转计数
146         } else if (result==0)
147         {
148             left_turn_count++;
149             right_turn_count = 0; // 重置左转计数
150         }
151         else {
152             std::cout << "result = " << result << std::endl;
153         }
154
155         // 当右转计数达到5次
156         if (right_turn_count == 5) {
157             std::cout << "B" << std::endl;
158             return true;
159         }
160
161         // 当左转计数达到5次
162         if (left_turn_count == 5) {
163             std::cout << "A" << std::endl;
164             return false;
165         }
166         cam0.read(input_frame0);
167     }
168     return false;
169 }
170
171 int check_blue_plain(){
172     cam0.read(input_frame0);
173     while (!input_frame0.empty()) {
174         if (blue_card_find(input_frame0)) {
175             return 0;
176         }
177         cam0.read(input_frame0);
178     }
179     return -1;
180 }

```

transform.hpp

```

1 #include <cmath>
2 #include <opencv2/opencv.hpp>
3

```

```

4 extern double measured_height;           //camera to the ground /millimeter
5 extern double measured_distance_near;    //camera to near point /millimeter
6 extern double measured_distance_far;     //camera to far point /millimeter
7 extern double measured_length_near;     //length of near line /millimeter
8 extern double measured_length_far;      //length of far line /millimeter
9
10 extern double center_angle; //rad
11 extern double bottom_angle; //rad
12 extern double tan_side_angle;
13 extern double depth;           //distance from view point to screen
14 extern cv::Point2d pos_map[60][320];
15
16 void initialize_math_info();

```

transform.cpp

```

1 #include <transform.hpp>
2
3 double measured_height      = 29.70;           //camera to the ground /centimeter
4 double measured_distance_near = 65.30;         //camera to near point /centimeter
5 double measured_distance_far  = 239.30;        //camera to far point /centimeter
6 double measured_length_near   = 123.70;        //length of near line /centimeter
7 double measured_length_far    = 375.20;        //length of far line /centimeter
8
9 double bottom_angle;
10 double center_angle;
11 double tan_side_angle;
12 double depth;
13 cv::Point2d pos_map[60][320];
14
15 void initialize_math_info(){
16     double middlex = 160, temp;
17     bottom_angle = acos(measured_height / measured_distance_near);
18     center_angle = acos(measured_height / measured_distance_far);
19     depth = 60 / tanf64(center_angle - bottom_angle);
20     tan_side_angle = tanf64((atan2f64(measured_length_near * 0.5, measured_distance_near)
21 + atan2f64(measured_length_far * 0.5, measured_distance_far)) * 0.5);
22
23     for (int i = 0; i < 60; i++) {
24         for (int j = 0; j < 320; j++) {
25             pos_map[i][j].y = measured_height * tanf64(center_angle - atan2f64(i, depth)
26 );
27             pos_map[i][j].x =
28 sqrt(pow(measured_height, 2) + pow(pos_map[i][j].y, 2))
29 / sqrt(pow(depth, 2) + pow(i, 2)) * (j - middlex);
30         }
31     }

```

undistort.hpp

```
1 #include <opencv2/opencv.hpp>
2
3 void undistort(const cv::Mat &frame);
```

undistort.cpp

```
1 #include "undistort.hpp"
2
3 cv::Mat K = (cv::Mat_<double>(3, 3) << 156.8597960958496, 0, 156.5480777481064,
4 0.0, 156.6560327205041, 127.6987415552855,
5 0.0, 0.0, 1.0),
6 D = (cv::Mat_<double>(1, 5) << -0.3495242471365975, 0.118682129385989, -0.0008891403272969765,
7 0.0008244924389640646, -0.01736317626728643),
8 mapx, mapy,
9 undistortedFrame, empty;
10
11 void undistort(const cv::Mat &frame) {
12     cv::initUndistortRectifyMap(K, D, empty, K, frame.size(), CV_32FC1, mapx, mapy);
13     cv::remap(frame, frame, mapx, mapy, cv::INTER_LINEAR);
14 }
```

utils.hpp

```
1 #include <opencv2/opencv.hpp>
2
3 #ifndef __INCLUDE_UTILS__
4 #define __INCLUDE_UTILS__
5
6 extern const int    servo_pin,
7 motor_pin,
8 cradlex_pin,
9 cradley_pin;
10
11 extern int    hough_result,
12 hough_threshold,
13 hough_length_limit,
14 normal_speed,
15 brake_speed,
16 brake_sleep,
17 change_track_turn_sleep,
18 change_track_straight_sleep,
19 turn_angle,
20 cross_cnt;
21
22 extern double    target_threshold,
23 kp,
```



```
24 kd,
25 threshold_l,
26 slow_speed,
27 left_threshold,
28 right_threshold;
29
30 struct Line{
31     double m, b;
32 };
33
34 extern cv::Mat input_frame0, input_frame2;
35 extern cv::Mat binary_frame2;
36 extern cv::VideoCapture cam0,
37 cam2;
38
39 extern cv::Ptr<cv::CLAHE> clahe;
40 extern Line left_real_line, right_real_line,
41 left_screen_line, right_screen_line;
42
43 extern char play_sound_command[1024];
44 extern bool auto_mode;
45 #endif
```

utils.cpp

```
1 #include "utils.hpp"
2
3 const int  servo_pin = 12,
4 motor_pin = 13,
5 cradlex_pin = 22,
6 cradley_pin = 23;
7
8 int hough_result,
9 hough_threshold,
10 hough_length_limit,
11 normal_speed,
12 brake_speed,
13 brake_sleep,
14 change_track_turn_sleep,
15 change_track_straight_sleep,
16 turn_angle,
17 cross_cnt;
18
19 double target_threshold,
20 kp,
21 kd,
22 threshold_l,
23 slow_speed,
24 left_threshold,
```

```

25 right_threshold;;
26
27 cv::Mat input_frame0,
28 input_frame2,
29 binary_frame2;
30
31 cv::VideoCapture cam0,
32 cam2;
33
34 cv::Ptr<cv::CLAHE> clahe;
35 Line left_real_line, right_real_line,
36 left_screen_line, right_screen_line;
37
38 char play_sound_command[1024];
39 bool auto_mode = true;

```

vector_control.cpp

```

1 #include "vector_control.hpp"
2
3 double last_error;
4 int pd;
5
6 void servo_control(double error) {
7     pd = -int((kp * error + kd * (error - last_error)) / 0.18) + 736;
8     last_error = error;
9     pd = std::min(std::max(pd, 570), 902);
10    gpioPWM(servo_pin, pd);
11 }

```

vector_control.hpp

```

1 #include <pigpio.h>
2 #include "utils.hpp"
3 #include <algorithm>
4
5 void servo_control(double error);

```

vector_fix.hpp

```

1 #include "get_lines.hpp"
2 #include <cmath>
3
4 double get_error_angle(Line &l_line, Line &r_line, double threshold);

```

vector_fix.cpp

```
1 #include "vector_fix.hpp"
2
3 double to_angle = 180 / CV_PI;
4 double get_error_angle(Line &l_line, Line &r_line, double threshold){
5     // x = y*m + b
6     double b = l_line.b + threshold * (r_line.b - l_line.b), k = r_line.m,
7     d = abs(b) / sqrt(1 + pow(k, 2)),
8     h2 = pow(d, 2) + pow(threshold_l, 2);
9     cv::Point2d vec(k, 1); vec *= sqrt(h2 / vec.ddot(vec)); vec.x += b;
10    return atan2f64(vec.x, vec.y) * to_angle;
11 }
```

参考文献

- [1] 张平. OpenCV 算法精解: 基于 Python 与 C++. 电子工业出版社, 2017. ISBN: 978-7-121-32495-6. 条码号: CB02894410
- [2] 李立宗. OpenCV 编程案例详解. 电子工业出版社, 2016. ISBN: 978-7-121-29952-0. 条码号: CB02778734
- [3] 肖铃. 学习 OpenCV4: 基于 Python 的算法实战. 电子工业出版社, 2022. ISBN: 978-7-121-42439-7. 条码号: CB03379875
- [4] Y. Danicl Liang. C++ 程序设计. 机械工业出版社, 2015. ISBN: 978-7-111-48514-8. 条码号: CB02592094